

MEDICAL DEVICE MARKETING

Importance

Medical device marketing connects **technology with healthcare needs**. A technically excellent device may fail commercially if it is too expensive, difficult to use, unavailable through healthcare channels, or does not provide sufficient clinical value.

2. Customers

A **customer** is a person, organization, or institution that purchases, uses, influences the purchase of, or benefits from a product or service.

In healthcare, the customer is not always the same person as the end user. For example, a patient may use a medical device, while the hospital purchases it and a doctor recommends it.

Types of customers in healthcare

1. Patient

The patient is the ultimate beneficiary of most healthcare products and services. Their comfort, safety, affordability, and treatment outcomes are important.

2. Doctor/Physician

Doctors may recommend, prescribe, evaluate, or influence the selection of medical devices.

3. Hospital

Hospitals are major institutional customers. They may purchase equipment such as patient monitors, ultrasound systems, ventilators, and diagnostic devices.

4. Biomedical Engineer

Biomedical engineers evaluate the technical performance, maintenance requirements, compatibility, and safety of medical equipment.

5. Procurement Department

The procurement team handles purchasing, quotations, vendor comparison, negotiations, and purchase orders.

6. Nurses and Technicians

They are important users of many medical devices and can influence purchasing decisions based on usability and reliability.

MEDICAL DEVICE MARKETING

7. Distributors and Dealers

They purchase products from manufacturers and distribute them to hospitals, clinics, pharmacies, and other customers.

Customer needs

Customer needs may include:

- Safety
- Clinical effectiveness
- Reliability
- Ease of use
- Affordability
- Durability
- Technical support
- Training
- After-sales service
- Availability of spare parts
- Regulatory compliance

Example

For a **portable ECG device**:

- Patient → receives the benefit
- Doctor → interprets the ECG
- Nurse → may operate the device
- Hospital → purchases the device
- Biomedical engineer → evaluates and maintains it
- Distributor → supplies the device

MEDICAL DEVICE MARKETING

Thus, understanding the **different roles of customers** is essential for successful medical device marketing.

Decision Making

Introduction

Decision making is the process of identifying a problem or opportunity, collecting relevant information, evaluating alternatives, selecting the best option, implementing the decision, and evaluating its outcome.

In healthcare and medical device marketing, decision making is complex because decisions may involve **patient safety, clinical effectiveness, cost, regulatory compliance, technology, and ethical considerations**.

Steps in the Decision-Making Process

1. Identify the problem

The first step is to clearly understand what problem needs to be solved.

Example:

A hospital experiences delays in detecting abnormal oxygen levels in patients.

2. Define the objective

The organization should determine what it wants to achieve.

For example:

- Improve patient monitoring
- Reduce response time
- Improve accuracy
- Reduce operating costs

3. Collect information

Relevant information is collected from:

- Doctors
- Nurses

MEDICAL DEVICE MARKETING

- Scalability
- Customer support
- **6. Evaluate alternatives**
- Each alternative is compared against the established criteria.
- For example:

Criteria	Device A	Device B	Device C
Cost	High	Medium	Low
Accuracy	High	High	Medium
Ease of use	Medium	High	High
Maintenance	High	Medium	Low
Connectivity	Medium	High	High

7. Select the best alternative

The alternative providing the best balance between **clinical value, technical performance, cost, safety, and customer requirements** is selected.

8. Implement the decision

The selected option is put into practice.

Implementation may include:

- Procurement
- Installation
- Training
- Testing
- Validation
- Documentation

9. Monitor the result

After implementation, performance should be monitored.

MEDICAL DEVICE MARKETING

- Ease of use
- Patient outcomes

Regulatory factors

- Regulatory approval
- Standards
- Quality requirements
- Documentation

Ethical factors

- Patient rights
- Privacy
- Informed consent
- Transparency

Organizational factors

- Budget
- Infrastructure
- Staff availability
- Management policy

Example

Suppose a hospital wants to purchase a new ultrasound machine. The decision-making process would involve:

Problem → Need identification → Market research → Identification of alternatives → Technical evaluation → Clinical evaluation → Cost comparison → Regulatory verification → Vendor evaluation → Purchase decision → Installation → Training → Performance evaluation.

MEDICAL DEVICE MARKETING

Buyer and Follow-Up

Buyer

A buyer is the person or organization responsible for purchasing a product or service. In healthcare, the buyer may be different from the actual user.

For example, a doctor may recommend a device, but the **hospital procurement department** may purchase it.

Types of healthcare buyers

1. Individual patients
2. Doctors and clinics
3. Hospitals
4. Government hospitals
5. Procurement departments
6. Distributors
7. Insurance organizations
8. Healthcare institutions

Buyer Decision Process

1. Identify the healthcare need
2. Search for available products
3. Compare alternatives
4. Evaluate technical and clinical performance
5. Evaluate price
6. Check regulatory approval
7. Negotiate with suppliers
8. Select the product
9. Purchase the product

MEDICAL DEVICE MARKETING

10. Evaluate the product after purchase

Follow-Up

Follow-up is the process of maintaining communication with a customer after initial contact, product demonstration, quotation, or purchase.

Importance of follow-up

- Builds customer relationships
- Identifies customer problems
- Provides technical support
- Improves customer satisfaction
- Encourages repeat purchases
- Helps identify new requirements
- Provides feedback for product improvement

Follow-up activities

Before purchase:

- Call the customer
- Provide product information
- Answer technical questions
- Provide quotation
- Arrange demonstrations

After purchase:

- Installation
- User training
- Performance verification
- Maintenance support
- Collect feedback

MEDICAL DEVICE MARKETING

- Handle complaints
- Provide warranty service

Example

After selling a patient monitor to a hospital, the company should contact the biomedical department to confirm installation, train users, check performance, and collect feedback.

Fundamentals of Reimbursement

Introduction

Healthcare reimbursement refers to the **payment made to healthcare providers, hospitals, clinics, or other healthcare organizations for providing healthcare services or products.**

It is an important component of healthcare economics because patients and healthcare providers may not directly bear the entire cost of treatment.

Major Participants

1. **Patient** – Receives healthcare.
2. **Healthcare provider** – Provides treatment or service.
3. **Hospital/Clinic** – Delivers healthcare services.
4. **Insurance company/payer** – May pay some or all eligible costs.
5. **Government** – Provides or finances healthcare through public schemes.
6. **Medical device company** – Supplies products used in treatment.

Basic Reimbursement Process

Patient receives treatment → Provider documents service → Claim is submitted → Payer evaluates claim → Claim is approved/rejected → Payment is made.

Types of Reimbursement

1. Fee-for-Service

Payment is made for individual healthcare services provided.

MEDICAL DEVICE MARKETING

2. Capitation

A fixed payment is provided for each patient for a defined period.

3. Bundled Payment

A single payment covers multiple services associated with a treatment or procedure.

4. Diagnosis-Related Payment

Payment is associated with the diagnosis or treatment category rather than each individual service.

5. Government-funded reimbursement

Government healthcare programs may reimburse hospitals or providers for eligible treatments.

Importance

Reimbursement affects:

- Product affordability
- Hospital purchasing decisions
- Patient access
- Medical device adoption
- Healthcare provider revenue
- Market potential

Example

A technologically advanced medical device may have excellent clinical performance but limited market adoption if patients or hospitals cannot afford it or if the relevant healthcare payment system does not cover its use.

MEDICAL DEVICE MARKETING

Healthcare Economics

Healthcare economics is the study of how **limited healthcare resources are allocated to satisfy the healthcare needs of individuals and society.**

Resources include:

- Money
- Medical equipment
- Healthcare professionals
- Medicines
- Hospital beds
- Infrastructure
- Time

Importance

Healthcare economics helps answer questions such as:

- Is the technology affordable?
- Does it provide sufficient clinical benefit?
- What is the cost of treatment?
- Is investment in the technology justified?
- How can limited healthcare resources be used efficiently?

Major concepts

Cost: Money or resources required for a healthcare product or service.

Benefit: Positive outcome obtained from healthcare intervention.

Cost-effectiveness: Comparison between cost and health outcome.

Cost-benefit: Comparison of monetary cost with monetary benefit.

Cost-utility: Comparison of cost with outcomes such as quality-adjusted life years.

Opportunity cost: The benefit lost when one option is selected instead of another.

MEDICAL DEVICE MARKETING

Example

A hospital may compare two patient-monitoring systems. One system may be cheaper initially, while another may have higher purchase cost but lower maintenance and better clinical performance. Healthcare economics helps determine which option provides greater overall value.

The Letter of Law, Ethics and Responsibilities

Introduction

Medical devices directly or indirectly affect human health and safety. Therefore, companies and healthcare professionals must comply with **laws, regulations, ethical principles, and professional responsibilities**.

The **letter of the law** refers to strict compliance with the actual legal requirements applicable to a product, organization, or activity. However, ethical responsibility goes beyond merely following minimum legal requirements.

A. Letter of Law

The letter of law means following established legal requirements exactly as specified by the relevant authorities.

For medical devices, legal requirements may cover:

- Product safety
- Quality management
- Manufacturing
- Clinical evaluation
- Regulatory approval
- Labelling
- Advertising
- Data protection
- Post-market surveillance

MEDICAL DEVICE MARKETING

- Reporting of adverse events

Why legal compliance is important

1. Protects patients
2. Ensures product safety
3. Builds customer confidence
4. Prevents legal penalties
5. Reduces product liability
6. Supports market acceptance
7. Protects the reputation of the organization

Example

A company should not market a medical device for a clinical indication that has not been legally authorized merely because the device appears technically capable of performing that function.

B. Ethics

Ethics refers to principles that guide people and organizations to distinguish between **right and wrong conduct**.

Important ethical principles include:

1. Beneficence

The product should provide benefit to the patient.

2. Non-maleficence

The product should not unnecessarily cause harm.

3. Autonomy

Patients should have the right to make informed decisions about their healthcare.

4. Justice

Healthcare resources and services should be provided fairly.

MEDICAL DEVICE MARKETING

5. Honesty

Manufacturers and healthcare professionals should provide accurate information.

6. Transparency

Limitations, risks, costs, and performance should not be deliberately hidden.

C. Responsibilities of Medical Device Companies

Product responsibility

The manufacturer should ensure:

- Safe design
- Reliable manufacturing
- Quality control
- Proper testing
- Appropriate documentation

Marketing responsibility

Companies should not:

- Make false claims
- Hide product limitations
- Mislead healthcare professionals
- Exaggerate clinical benefits
- Provide unsupported performance claims

Patient responsibility

Patient safety should remain a primary consideration.

Environmental responsibility

Companies should consider:

- Safe disposal

MEDICAL DEVICE MARKETING

- Electronic waste
- Sustainable materials
- Energy consumption
- Environmental impact

D. Responsibilities of Healthcare Professionals

Healthcare professionals should:

- Prioritize patient welfare
- Maintain confidentiality
- Use devices appropriately
- Provide accurate information
- Report adverse events
- Maintain professional competence
- Avoid conflicts of interest

E. Conflict of Interest

A conflict of interest occurs when personal, financial, or organizational interests may influence professional judgment.

Example:

A healthcare professional recommending a particular device because they receive an undisclosed financial benefit from the manufacturer represents an ethical concern.

F. Ethical Marketing

Ethical marketing requires:

1. Truthful product information
2. Evidence-based claims
3. Transparent pricing
4. No misleading advertisements

MEDICAL DEVICE MARKETING

5. Respect for patient privacy
6. Responsible promotion
7. Proper disclosure of risks
8. No inappropriate influence on purchasing decisions

Law vs Ethics

Law	Ethics
Legally mandatory	Based on moral/professional principles
Defined by legal authorities	Influenced by professional and societal values
Violation may result in legal penalties	Violation may damage trust and reputation
Represents minimum legal requirements	May demand a higher standard of conduct

Business Plan

Introduction

A business plan is a structured document that explains **what a business intends to achieve, how it will achieve it, who its customers are, what resources are required, and whether the business is financially viable.**

For a medical device company, a business plan connects the **healthcare problem → product → market → customer → financial model → implementation strategy.**

A typical business plan includes:

1. Executive summary
2. Problem identification
3. Product/service description
4. Customer identification
5. Market research
6. Competitor analysis

MEDICAL DEVICE MARKETING

7. Opportunity assessment
8. Marketing strategy
9. Financial analysis
10. Project scope
11. Risk analysis
12. Implementation plan
13. Stage-gate process

Market Research

Market research is the systematic process of collecting and analyzing information about **customers, competitors, market demand, pricing, trends, and business opportunities.**

Objectives

- Understand customer needs
- Estimate market size
- Identify competitors
- Determine customer preferences
- Understand pricing
- Identify market gaps
- Reduce business risk

Types of Market Research

Primary research

Information collected directly from customers.

Examples:

- Interviews
- Questionnaires
- Surveys

MEDICAL DEVICE MARKETING

- Focus groups
- Product demonstrations
- Hospital visits

Secondary research

Information obtained from existing sources.

Examples:

- Research papers
- Government reports
- Industry reports
- Published statistics
- Company websites
- Market databases

Market Research Process

Identify problem → Define research objective → Collect information → Analyze data →

Identify market opportunity → Make business decision

Example

For an **IoT-based sepsis monitoring device**, market research may determine:

- Number of hospitals requiring continuous monitoring
- Existing monitoring systems
- Doctors' requirements
- Device cost expectations
- Competitor products
- Required regulatory approvals
- Potential customers

MEDICAL DEVICE MARKETING

Accessing Opportunity / Assessing Opportunity

Assessing an opportunity means evaluating whether a particular idea or market need has sufficient **technical, clinical, commercial, and financial potential**.

Major factors

1. Problem

Is there a genuine healthcare problem?

2. Market size

How many potential customers exist?

3. Customer need

Are customers willing to use or purchase the solution?

4. Competition

Are similar products already available?

5. Competitive advantage

What makes the proposed product better?

6. Technical feasibility

Can the product actually be developed?

7. Regulatory feasibility

Can the product obtain necessary approvals?

8. Financial feasibility

Can the company develop and sell the product profitably?

9. Scalability

Can production be increased when demand grows?

10. Risk

What technical, financial, clinical, regulatory, and market risks exist?

Opportunity Assessment Example

MEDICAL DEVICE MARKETING

For a smart prosthetic socket:

Problem: Conventional sockets may cause discomfort.

Opportunity: Develop a sensor-based socket that monitors pressure and fit.

Customers: Amputees, rehabilitation centers, hospitals.

Technology: Pressure sensors + IoT + data analysis.

Market advantage: Real-time monitoring and personalized fitting.

Feasibility: Requires technical validation, clinical testing, manufacturing capability, and regulatory compliance.

Financial Analysis

Financial analysis evaluates whether a proposed business or project is **economically viable and financially sustainable**.

Major components

1. Initial investment

Money required to start the project.

Examples:

- Equipment
- Software
- Manufacturing setup
- Research and development
- Regulatory expenses
- Infrastructure

2. Operating costs

Costs required for regular operation.

Examples:

MEDICAL DEVICE MARKETING

- Salaries
- Electricity
- Maintenance
- Raw materials
- Marketing
- Transportation

3. Revenue

Income generated through product sales or services.

$$\text{Revenue} = \text{Selling Price} \times \text{Number of Units Sold}$$

4. Profit

$$\text{Profit} = \text{Revenue} - \text{Total Cost}$$

5. Break-even analysis

Break-even point is the point where total revenue equals total cost.

$$\text{Break-even units} = \text{Fixed Cost} \div (\text{Selling Price} - \text{Variable Cost per Unit})$$

6. Return on Investment

ROI measures the return obtained from an investment.

$$\text{ROI} = (\text{Net Return} \div \text{Investment}) \times 100$$

7. Cash flow

Cash flow represents the movement of money into and out of the business.

8. Financial risk

Possible risks include:

- Low sales
- Increased manufacturing cost
- Delayed regulatory approval

MEDICAL DEVICE MARKETING

- High R&D expenditure
- Competition
- Supply-chain problems

Example

If a medical device costs ₹10,000 to manufacture and is sold for ₹18,000, the difference contributes toward covering fixed costs and generating profit.

Project Scope

Project scope defines **what a project will and will not cover**.

It establishes the project's boundaries and prevents uncontrolled expansion of activities.

Components of project scope

1. Project objective
2. Deliverables
3. Requirements
4. Activities
5. Timeline
6. Resources
7. Budget
8. Responsibilities
9. Constraints
10. Exclusions
11. Acceptance criteria

Example

For an IoT patient monitoring project:

Objective: Develop a system for real-time monitoring of vital parameters.

MEDICAL DEVICE MARKETING

Inputs: Heart rate, SpO₂, temperature, etc.

Technology: Sensors + microcontroller + communication + cloud platform.

Output: Real-time display and alerts.

Deliverable: Functional prototype and validated monitoring system.

Exclusion: Full commercial regulatory approval may fall outside the initial prototype-development phase.

Importance

A clearly defined project scope:

- Controls cost
- Controls time
- Defines responsibilities
- Prevents scope creep
- Helps measure progress
- Improves project management

Stage Gate Process

Introduction

The **Stage-Gate process** is a structured product development methodology in which a project progresses through a series of **stages separated by decision gates**.

Each stage performs specific activities, while each gate evaluates whether the project should:

- Continue
- Stop
- Be modified
- Be delayed
- Be redirected

MEDICAL DEVICE MARKETING

It is particularly useful in medical device development because healthcare products involve significant **technical, clinical, regulatory, financial, and market risks**.

General Stage-Gate Model

Idea → Gate 1 → Concept → Gate 2 → Development → Gate 3 → Testing/Validation → Gate 4 → Launch → Gate 5 → Post-launch review

Stage 1 – Idea Generation

Potential product ideas are identified.

Sources include:

- Customer complaints
- Clinical problems
- Doctor feedback
- Research
- New technologies
- Market gaps

Gate 1: Determine whether the idea deserves further investigation.

Stage 2 – Concept Development

The selected idea is converted into a preliminary product concept.

Activities include:

- Customer identification
- Market research
- Competitor analysis
- Technical feasibility
- Initial risk assessment
- Preliminary business case

Gate 2: Decide whether the concept is sufficiently attractive and feasible.

MEDICAL DEVICE MARKETING

Stage 3 – Product Development

The actual product is designed and developed.

Activities include:

- Engineering design
- Prototype development
- Software development
- Component selection
- Risk management
- Design documentation

Gate 3: Evaluate whether the product is ready for testing and validation.

Stage 4 – Testing and Validation

The developed product is tested to determine whether it meets requirements.

Testing may include:

- Functional testing
- Safety testing
- Performance testing
- Usability testing
- Verification
- Validation
- Clinical evaluation where applicable

Gate 4: Decide whether the product is ready for commercialization.

Stage 5 – Product Launch

The product is introduced into the market.

Activities include:

MEDICAL DEVICE MARKETING

- Regulatory compliance
- Manufacturing
- Packaging
- Marketing
- Sales
- Distribution
- Customer training

Gate 5: Evaluate commercial performance.

Stage 6 – Post-Launch Review

After commercialization, the company evaluates:

- Sales
- Customer satisfaction
- Product complaints
- Clinical performance
- Maintenance
- Profitability
- Adverse events
- Opportunities for improvement

Decision Criteria at Gates

- Each gate may consider:

Factor	Question
Market	Is there sufficient demand?
Customer	Does it solve a real customer problem?
Technical	Can it be developed successfully?
Clinical	Does it provide meaningful healthcare benefit?

MEDICAL DEVICE MARKETING

Regulatory	Can it satisfy applicable requirements?
Financial	Is it economically viable?
Risk	Are risks acceptable?
Competition	Does it have sufficient competitive advantage?

Advantages of Stage-Gate Process

1. Reduces risk

Problems can be identified before large investments are made.

2. Controls expenditure

Projects that are unlikely to succeed can be stopped early.

3. Improves product quality

Each stage has defined requirements and evaluation criteria.

4. Supports informed decisions

Management receives structured information before approving the next stage.

5. Improves resource allocation

Money, people, equipment, and time can be directed toward promising projects.

6. Supports regulatory planning

Medical device regulatory and quality requirements can be considered throughout development.

7. Improves commercialization

Market and customer requirements are considered before product launch.

Example – Smart Patient Monitoring Device

Idea: Develop a wearable device for continuous monitoring.



Concept: Identify parameters such as heart rate, SpO₂, temperature, and activity.



Development: Develop sensors, embedded system, communication, and software.



Testing: Verify accuracy, safety, usability, and connectivity.



Regulatory/Commercial Gate: Evaluate compliance and business viability.



Launch: Manufacture and sell the product.



MEDICAL DEVICE MARKETING

Post-launch: Collect customer feedback and monitor product performance.



KALASALINGAM
ACADEMY OF RESEARCH AND EDUCATION
(DEEMED TO BE UNIVERSITY)

Under sec. 3 of UGC Act 1956. Accredited by NAAC with "A" Grade



JAVA PROGRAMMING

BY

Dr.T.Dhiliphan Rajkumar

Course Outline



CO 1. Understand the object oriented concepts using java



CO 2. Apply Object Oriented Concepts to develop Java Application



CO 3. Identify, Formulate and Analyze a Real World Problem and Develop a solution using Java Programming Concepts



CO 4. Ability to Work individually as well as in team to solve problem



CO 5. Communicate effectively with technical community during Interviews

Course description

Java still has its importance and popularity in the software industry, in spite of recent development of several high level languages. It has an excellent support of high-level as well as low-level functionality, which makes it suitable for many applications. This course provides students with a comprehensive study of the Java programming language. Instead of Classroom lectures, practical classes stress the strengths of Java, which provide the students with the means of writing efficient, maintainable, and portable code. A course with mini project is one of the great learning opportunities to develop students in various aspects. This course will also help the students during their placement sittings, as most of the companies test proficiency in programming using Java.

UNIT I OOP IN JAVA & INHERITANCE

Object Oriented Programming Concepts - OOP in Java - Characteristics of Java -Fundamental Programming Structures in Java - Defining classes in Java - Comments, Data Types, Variables, Operators, Control Flow, Arrays - constructors, methods -access specifiers - static members - Packages- Inheritance - Super classes- sub classes -Protected members - constructors in sub classes- Strings.

UNIT II EXCEPTION HANDLING AND I/O

Exceptions - exception hierarchy - throwing and catching exceptions - built-in exceptions, creating own exceptions, Stack Trace Elements-Input / Output Basics - Streams - Byte streams and Character streams - Reading and Writing Console - Reading and Writing Files - abstract classes and methods - final methods and classes

UNIT III INTERFACES AND MULTITHREADING

Interfaces - defining an interface, implementing interface, differences between classes and interfaces - extending interfaces - Object cloning - inner classes-Differences between multi-threading and multitasking, thread life cycle, creating threads, synchronizing threads, Inter-thread communication, daemon threads - Generic Programming.

UNIT IV AWT AND EVENT DRIVEN PROGRAMMING

AWT Event Hierarchy- Components - Graphics programming - Applets-Frame -working with 2D shapes - Using color, fonts, and images - Basics of event handling - event handlers - adapter classes - actions - mouse events - Introduction to Swing - layout management - Swing Components - Windows -Menus - Dialog Boxes.

UNIT V NETWORKING AND JDBC

Networking Basics - The Networking Classes and Interfaces -TCP/IP Client Sockets URL TCP/IP Server Sockets - **Datagrams** - A Relational Database Overview - JDBC Introduction - JDBC Product Components - JDBC Architecture - Case studies.



Unit II

Outcomes

- Handle exceptions
- Create own Exception handlers
- Get Input from command line
- Read/Write I/O using files
- Use Abstract and Final Methods and Classes

Unit 2 Outline



Lesson 1. Exceptions



Lesson 2. Built In and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Describes the exception handling mechanisms, different types of exceptions and their hierarchy, way to obtain command line arguments as inputs both numbers and characters, programming for files and abstract & final keywords.

Course Progress (use before starting new lesson)

Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Motivation

- We seek robust programs
- When something unexpected occurs
 - Ensure program detects the problem
 - Then program must do something about it
- Extensive testing of special situations can result in "spaghetti code"
- Need mechanism to check for problem where it could occur
- When condition does occur
 - Have control passed to code to handle the problem

Exceptions

The exception handling in java is one of the powerful mechanism to handle the runtime errors so that normal flow of the application can be maintained.

An exception is an event, which occurs during the execution of a program that interrupts the normal flow of the program. It is an error thrown by a class or method reporting an error in code.

The process of converting system error messages into user friendly error message is known as **Exception handling**.

This is one of the powerful features of Java to handle run time error and maintain normal flow of java application.



Exception and Exception handling

Exception

Exception is an abnormal condition. In java, exception is an event that disrupts the normal flow of the program. It is an object which is thrown at runtime.

Exception handling

Exception Handling is a mechanism to handle runtime errors such as ClassNotFoundException, IO, SQL, Remote etc.

Advantage of Exception Handling

The core advantage of exception handling is to maintain the normal flow of the **application**. Exception normally disrupts the normal flow of the application that is why we use exception handling. Let's take a scenario:

statement 1;

statement 2; //exception occurs

statement 3;

statement 4;

statement 5;

Suppose there is 5 statements in your program and there occurs an exception at statement 2, rest of the code will not be executed i.e. statement 3 to 5 will not run. If we perform exception handling, rest of the statement will be executed. That is why we use exception handling in java.

When an exception can occur?

Exception can occur in two ways

- at runtime (known as runtime exceptions)
- at compile-time (known as Compile-time exceptions).

Reasons for Exceptions

There can be several reasons for an exception. For example, following situations can cause an exception

- Opening a non-existing file
- Network connection problem
- Operands being manipulated are out of prescribed ranges

Exception Handling

- When an exceptional condition arises, an object representing that exception is created and thrown in the method that caused the error.
- That method may choose to handle the exception itself, or pass it on. Either way, at some point, the exception is caught and processed.
- Java exception handling is managed via five keywords: **try**, **catch**, **throw**, **throws**, and **finally**.
- Program statements that you want to monitor for exceptions are contained within a **try block**. If an exception occurs within the try block, it is thrown.
- Your code can **catch** this exception (using catch) and handle it in some rational manner. System-generated exceptions are automatically thrown by the Java run-time system.

- To manually throw an exception, use the keyword **throw**. Any exception that is thrown out of a method must be specified as such by a **throws** clause.
- Any code that absolutely must be executed after a try block completes is put in a **finally** block.
- Code within try/catch block is referred as protected code. Syntax is as follows

```
try { // Protected code }  
catch (ExceptionName e1)  
    { // Catch block }  
Finally ( // block to be executed at end }
```

Throwing and Catching Exceptions

- The code which is prone to exceptions is placed in the try block.
- When an exception occurs, that exception occurred is handled by catch block associated with it.
- Every try block should be immediately followed either by a catch block or finally block.
- A catch statement involves declaring the type of exception you are trying to catch.
- If an exception occurs in protected code, the catch block (or blocks) that follows the try is checked.
- If the type of exception that occurred is listed in a catch block, the exception is passed to the catch block much as an argument is passed into a method parameter.

Single Catch

```
class Exc2 {  
    public static void main( String args []) { int d, a;  
    try { // monitor a block of code.  
        d = 0;  
        a = 42 / d;  
        System . out. println (" This will not be printed .");  
    }  
    catch ( ArithmeticException e) // divide - by- zero error  
    {  
        System . out. println (" Division by zero.");  
    }  
    System . out. println (" After catch statement.");  
    }  
}
```

Using Multiple Catch

- In some cases, more than one exception could be raised by a single piece of code. To handle this type of situation, you can specify two or more catch clauses, each catching a different type of exception.
- When an exception is thrown, each catch statement is inspected in order, and the first one whose type matches that of the exception is executed.
- After one catch statement executes, the others are bypassed, and execution continues after the try / catch block.

Example of Multiple Catch

```
class MultipleCatches
{
    public static void main( String args [])
    { try {
        int a = args. length ;

        System . out. println (" a  = " + a);  int b = 42 / a;

        int c[] = { 1 };  c[42] = 99;
    }
    catch ( ArithmeticException e)
    {
        System . out. println (" Divide by 0: " + e);
    }

    catch ( Array Index Out OfBounds Exception e)
    {

        System . out. println (" Array index oob: " + e);
    }
    System . out. println (" After try/ catch blocks.");
}
```

Usage of throw

- It is possible for your program to throw an exception explicitly, using the **throw** statement.
- The general form of throw is shown here: **throw ThrowableInstance;**
- Here, ThrowableInstance must be an object of type Throwable or a subclass of Throwable.
- Primitive types, such as int or char, as well as non-Throwable classes, such as String and Object, cannot be used as exceptions.
- The flow of execution stops immediately after the throw statement; any subsequent statements are not executed.
- The nearest enclosing try block is inspected to see if it has a catch statement that matches the type of exception.
- If it does find a match, control is transferred to that statement. If not, then the next enclosing try statement is inspected, and so on.
- If no matching catch is found, then the default exception handler halts the program and prints the stack trace.

Usage of throw - Example

```
class Throw Demo {  
    static void demoproc () { try {  
        throw new NullPointerException (" demo");  
    } catch ( NullPointerException e) { System . out. println (" Caught inside demoproc."); throw  
e; // rethrow the exception  
    }  
}  
    public static void main( String args []) { try {  
        demoproc ();  
    } catch ( NullPointerException e) { System . out. println (" Recaught: " + e);  
    }  
}
```

Usage of throws

- If a method is capable of causing an exception that it does not handle, it must specify this behavior so that callers of the method can guard themselves against that exception.
- You do this by including a throws clause in the method's declaration.
- A throws clause lists the types of exceptions that a method might throw.
- This is necessary for all exceptions, except those of type Error or RuntimeException, or any of their subclasses.
- All other exceptions that a method can throw must be declared in the throws clause.
- If they are not, a compile-time error will result.
- This is the general form of a method declaration that includes a throws clause:

```
type method-name(parameter-list) throws exception-list  
{  
// body of method  
}
```

- Here, exception-list is a comma-separated list of the exceptions that a method can throw.

Usage of throws - Example

```
class ThrowsDemo {  
    static void throw One () throws IllegalAccessException { System . out. println (" Inside throw  
One.");  
    throw new IllegalAccessException (" demo");  
}  
  
    public static void main( String args []) { try {  
        throw One ();  
    } catch ( IllegalAccessException e) { System . out. println (" Caught " + e);  
    }  
    }  
}
```

Usage of finally

- **finally** creates a block of code that will be executed after a try /catch block has completed and before the code following the try/catch block.
- The finally block will execute whether or not an exception is thrown.
- If an exception is thrown, the finally block will execute even if no catch statement matches the exception.

Example of finally



```
class Finally Demo { static void procA () {  
try {  
  
System . out . println (" inside procA "); throw new RuntimeException (" demo");  
} finally { System . out . println (" procA ' s finally ");  
}  
  
static void procB () { try {  
  
System . out . println (" inside procB "); return ;  
} finally { System . out . println (" procB ' s finally ");  
}  
  
public static void main( String args []) { try {  
procA ();  
} catch ( Exception e) { System . out . println (" Exception caught"); procB ();  
}}  
}
```

Topic 1
Exceptions

Topic 2
Exception Handling

Topic 3
Throwing and Catching Exceptions

Topic 4
Multiple Catch

Topic 5
Usage of throw, throws and finally

First Lesson Summary

Learned about Exception handling and their related terms

Course Progress (use before starting new lesson)



Lesson 1. Exceptions

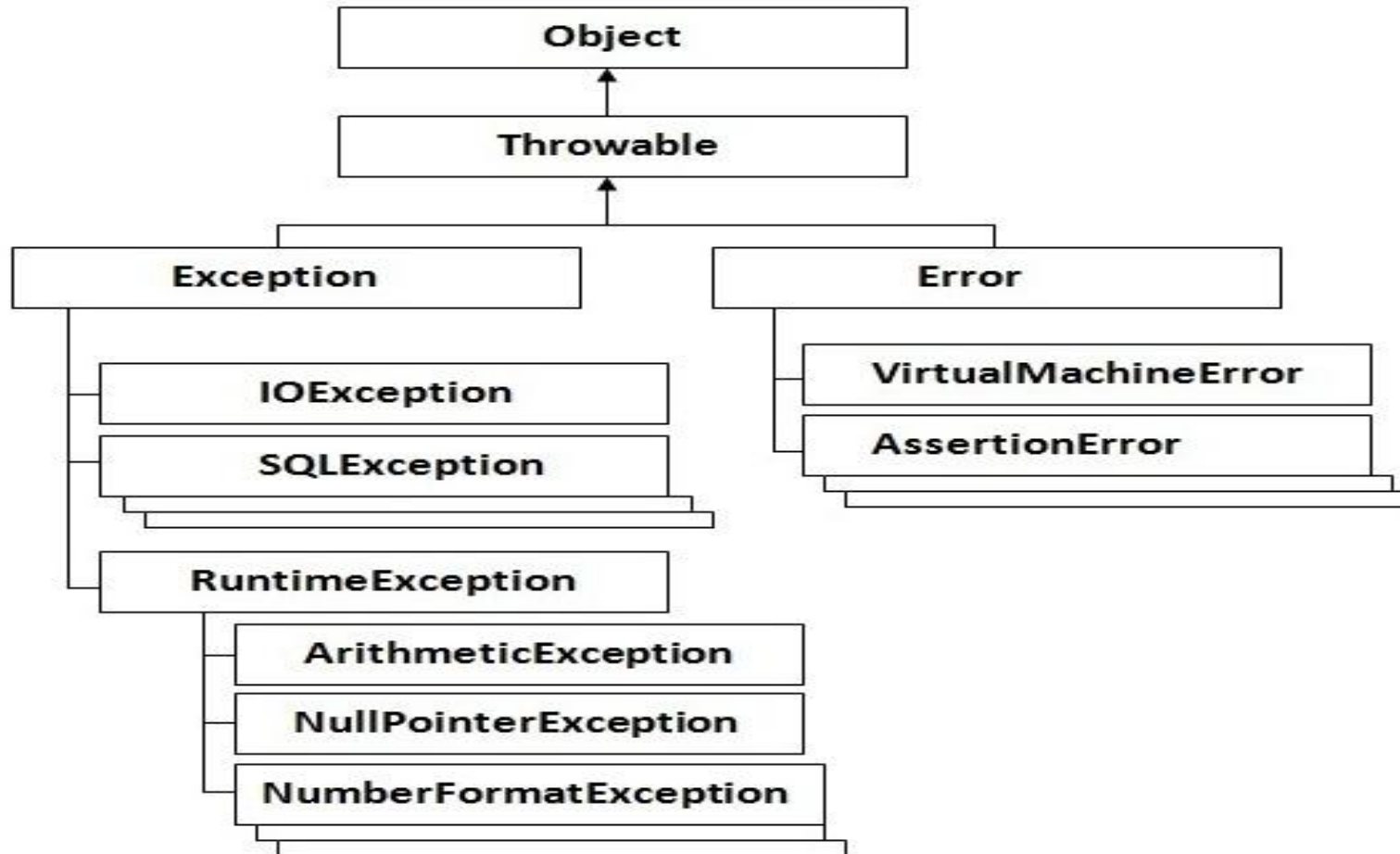
Lesson 2. Built in and Custom Exceptions

Lesson 3. Byte Stream

Lesson 4. Character Stream

Lesson 5. Abstract and Final

Hierarchy of Java Exception classes

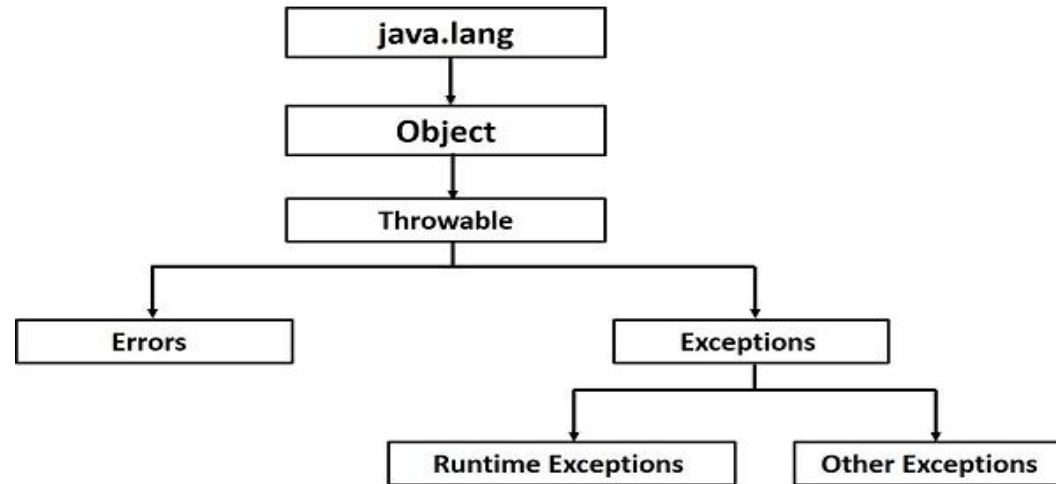


Types of Exceptions

There are mainly two types of exceptions: checked and unchecked where error is considered as unchecked exception. The sun microsystem says there are three types of exceptions:

- Checked Exception
- Unchecked Exception
- Error

Hierarchy of Java Exception



Checked, Unchecked and Error

1) Checked Exception

The classes that extend Throwable class except RuntimeException and Error are known as checked exceptions e.g. IOException, SQLException etc. Checked exceptions are checked at compile-time.

2) Unchecked Exception

The classes that extend RuntimeException are known as unchecked exceptions e.g. ArithmeticException, NullPointerException, ArrayIndexOutOfBoundsException etc. Unchecked exceptions are not checked at compile-time rather they are checked at runtime.

3) Error

Error is irrecoverable e.g. OutOfMemoryError, VirtualMachineError, AssertionError etc.

Common scenarios where exceptions may occur



1) Scenario where ArithmeticException occurs

If we divide any number by zero, there occurs an ArithmeticException.

```
int a=50/0;//ArithmeticException
```

2) Scenario where NullPointerException occurs

If we have null value in any variable, performing any operation by the variable occurs an NullPointerException.

```
String s=null;
```

```
System.out.println(s.length());//NullPointerException
```

Common scenarios Con”

3) Scenario where NumberFormatException occurs

The wrong formatting of any value, may occur NumberFormatException. Suppose I have a string variable that have characters, converting this variable into digit will occur NumberFormatException.

```
String s="abc";
```

```
int i=Integer.parseInt(s); //NumberFormatException
```

4) Scenario where ArrayIndexOutOfBoundsException occurs

If you are inserting any value in the wrong index, it would result ArrayIndexOutOfBoundsException as shown below:

```
int a[]=new int[5];
```

```
a[10]=50; //ArrayIndexOutOfBoundsException
```

Arithmetic exception

```
class ArithmeticException_Demo {  
    public static void main(String args[])  
    {  
        try {  
            int a = 30, b = 0;  
            int c = a / b; // cannot divide by zero  
            System.out.println("Result = " + c);  
        }  
        catch (ArithmeticException e) {  
            System.out.println("Can't divide a number by 0");  
        }  
    }  
}
```

Output

Can't divide a number by 0

ArrayIndexOutOfBoundsException Exception

```
class ArrayIndexOutOfBounds_Demo {  
    public static void main(String args[])  
    {  
        try {  
            int a[] = new int[5];  
            a[6] = 9; // accessing 7th element in an array of  
                // size 5  
        }  
        catch (ArrayIndexOutOfBoundsException e) {  
            System.out.println("Array Index is Out Of Bounds");  
        }  
    }  
}
```

Output

Array Index is Out Of Bounds

ClassNotFoundException

```
class B
{

}

class G
{

}

class MyClass {
public static void main(String[] args)
{
    Object o = class.forName(args[0]).newInstance();
    System.out.println("Class created for" + o.getClass().getName());
}
}
```

Output

ClassNotFoundException

Other standard Exceptions available

FileNotFoundException

IOException

InterruptedException

NoSuchMethodException

NullPointerException

NumberFormatException

StringIndexOutOfBoundsException

ClassCastException

StackOverflowError

NoClassDefFoundError

IllegalArgumentException

Exceptions Methods



Important Methods available in throwable class

- **public String getMessage()**

Returns a detailed message about the exception that has occurred. This message is initialized in the Throwable constructor.

- **public Throwable getCause()**

Returns the cause of the exception as represented by a Throwable object.

- **public String toString()**

Returns the name of the class concatenated with the result of getMessage().

Exceptions Methods Con”

- **public void printStackTrace()**

Prints the result of toString() along with the stack trace to System.err, the error output stream.

- **public StackTraceElement [] getStackTrace()**

Returns an array containing each element on the stack trace. The element at index 0 represents the top of the call stack, and the last element in the array represents the method at the bottom of the call stack.

- **public Throwable fillInStackTrace()**

Fills the stack trace of this Throwable object with the current stack trace, adding to any previous information in the stack trace.

Exception Hierarchy

All exception types are subclasses of the built-in class **Throwable**. Thus, **Throwable** is at the top of the exception class hierarchy. Immediately below **Throwable** are two subclasses that partition exceptions into two distinct branches.

One branch is headed by **Exception**.

- This class is used for exceptional conditions that user programs should catch.
- This is also the class that you will subclass to create your own custom exception types.
- There is an important subclass of **Exception**, called **RuntimeException**.
- Exceptions of this type are automatically defined for the programs that you write and include things such as division by zero and invalid array indexing.

Exception Hierarchy Con”



The other branch is topped by **Error**, which defines exceptions that are not expected to be caught under normal circumstances by your program.

- Exceptions of type **Error** are used by the Java run-time system to indicate errors having to do with the run-time environment, itself.
- Stack overflow is an example of such an error.

Built in Exceptions

Exception	Meaning
ArithmeticException	Arithmetic error, such as divide-by-zero.
ArrayIndexOutOfBoundsException	Array index is out-of-bounds.
ArrayStoreException	Assignment to an array element of an incompatible type.
ClassCastException	Invalid cast.
EnumConstantNotPresentException	An attempt is made to use an undefined enumeration value.
IllegalArgumentException	Illegal argument used to invoke a method.
IllegalMonitorStateException	Illegal monitor operation, such as waiting on an unlocked thread.
IllegalStateException	Environment or application is in incorrect state.
IllegalThreadStateException	Requested operation not compatible with current thread state.
IndexOutOfBoundsException	Some type of index is out-of-bounds.
NegativeArraySizeException	Array created with a negative size.
NullPointerException	Invalid use of a null reference.
NumberFormatException	Invalid conversion of a string to a numeric format.
SecurityException	Attempt to violate security.
StringIndexOutOfBoundsException	Attempt to index outside the bounds of a string.
TypeNotPresentException	Type not found.
UnsupportedOperationException	An unsupported operation was encountered.

Built in Exceptions

Exception	Meaning
ClassNotFoundException	Class not found.
CloneNotSupportedException	Attempt to clone an object that does not implement the Cloneable interface.
IllegalAccessException	Access to a class is denied.
InstantiationException	Attempt to create an object of an abstract class or interface.
InterruptedException	One thread has been interrupted by another thread.
NoSuchFieldException	A requested field does not exist.
NoSuchMethodException	A requested method does not exist.
ReflectiveOperationException	Superclass of reflection-related exceptions. (Added by JDK 7.)

Custom Exceptions

- To create your own exception types to handle situations specific to your applications: just define a subclass of Exception.
- Your subclasses don't need to actually implement anything—it is their existence in the type system that allows you to use them as exceptions.
- The Exception class does not define any methods of its own.
- It does, of course, inherit those methods provided by Throwable.
- Example while entering a if greater than 10 throws an exception

Custom Exceptions Example

```
class My Exception extends Exception {  
    private int detail;  
    My Exception ( int a) { detail = a; }  
    public String toString () {  
        return " My Exception [" + detail + "]; }  
    }  
class Exception Demo {  
    static void compute( int a) throws My Exception {  
        System . out. println (" Called compute(" + a + ")");  
        if( a > 10) throw new My Exception ( a);  
        System . out. println (" Normal exit");  
    }  
    public static void main( String args []) {  
        try { compute (1); compute (20);}  
        catch ( My Exception e) { System . out. println (" Caught " + e); }  
    }  
}
```

User-defined Custom Exception in Java

```
// A Class that represents use-defined exception
class MyException extends Exception
{
    public MyException(String s)
    {
        // Call constructor of parent Exception
        super(s);
    }
}

// A Class that uses above MyException
public class Main
{
    // Driver Program
    public static void main(String args[])
    {
        try
        {
            // Throw an object of user defined
            throw new MyException("Hello");
        }
        catch (MyException ex)
        {
            System.out.println("Caught");

            // Print the message from MyException
            System.out.println(ex.getMessage());
        }
    }
}
```

exception

object
Output
Caught
Hello

Topic 1
Types of Exceptions

Topic 2
Exception Methods

Topic 3
Exception Hierarchy

Topic 4
Built in Exceptions

Topic 5
Custom Exceptions

Second Lesson Summary

Understood and Use different types of built in Exceptions and custom exceptions

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Streams



Lesson 4. Character Streams

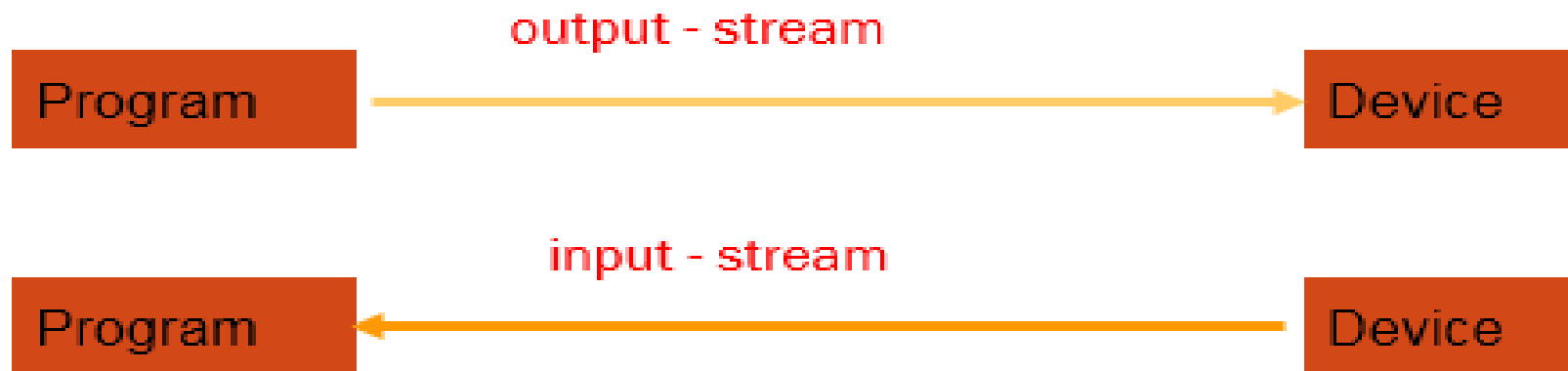


Lesson 5. Abstract and Final

I/O



- Usual Purpose: storing data to ‘nonvolatile’ devices, e.g. harddisk
- Classes provided by package java.io
- Data is transferred to devices by ‘streams’

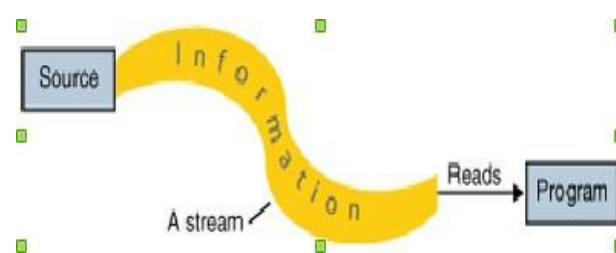


Stream

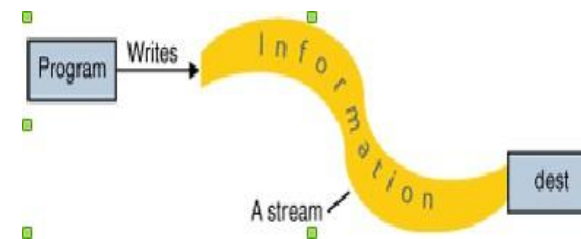
Java programs perform I/O through streams.

- **Stream:** is an abstraction that either produces or consumes information. A stream is linked to a physical device by the Java I/O system.
- **Input stream** can abstract many different kinds of input: **from a disk file, a keyboard, or a network socket.**
- **Output stream** may refer to the **console, a disk file, or a network connection.**

Java implements streams within class hierarchies defined in the **java.io** package.



Input stream



Output stream

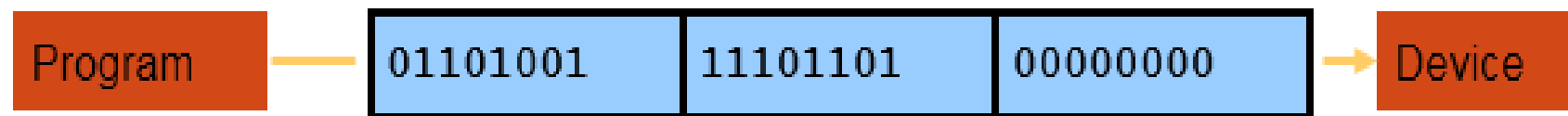
Overview of I/O streams

- The java.io package contains a collection of stream classes that support algorithms for reading and writing.
- To use these classes, a program needs to import the java.io package.
- The stream classes are divided into two class hierarchies, based on the data type (either characters or bytes) on which they operate
 - i.e Character Stream Classes
 - Byte Stream Classes

JAVA distinguishes between 2 types of streams:



- Binary Streams, containing 8 - bit information



I/O Streams Having



I/O Streams Having

- 2 types of streams (text / binary)
- 2 directions (input / output)

Results in 4 base-classes dealing with I/O:

1. InputStream: byte-input
2. OutputStream: byte-output
3. Reader: text-input
4. Writer: text-output

Streams

Java defines two types of streams: **byte** and **character**.

- **Byte streams:** provide a convenient means for handling input and output of bytes. Byte streams are used, for example, when reading or writing binary data.
- **Character streams:** provide a convenient means for handling input and output of characters. They use Unicode and, therefore, can be internationalized. Also, in some cases, character streams are more efficient than byte streams.
- **Byte streams** are defined by using two class hierarchies. **InputStream** and **OutputStream**.
- **Character streams** are defined by using two class hierarchies. **Reader** and **Writer**.

Byte Stream Classes

Stream Class	Meaning
BufferedInputStream	Buffered input stream
BufferedOutputStream	Buffered output stream
ByteArrayInputStream	Input stream that reads from a byte array
ByteArrayOutputStream	Output stream that writes to a byte array
DataInputStream	An input stream that contains methods for reading the Java standard data types
DataOutputStream	An output stream that contains methods for writing the Java standard data types
FileInputStream	Input stream that reads from a file
FileOutputStream	Output stream that writes to a file
FilterInputStream	Implements InputStream
FilterOutputStream	Implements OutputStream
InputStream	Abstract class that describes stream input
ObjectInputStream	Input stream for objects
ObjectOutputStream	Output stream for objects
OutputStream	Abstract class that describes stream output
PipedInputStream	Input pipe
PipedOutputStream	Output pipe
PrintStream	Output stream that contains print() and println()
PushbackInputStream	Input stream that supports one-byte “unget,” which returns a byte to the input stream
SequenceInputStream	Input stream that is a combination of two or more input streams that will be read sequentially, one after the other

InputStream vs OutputStream



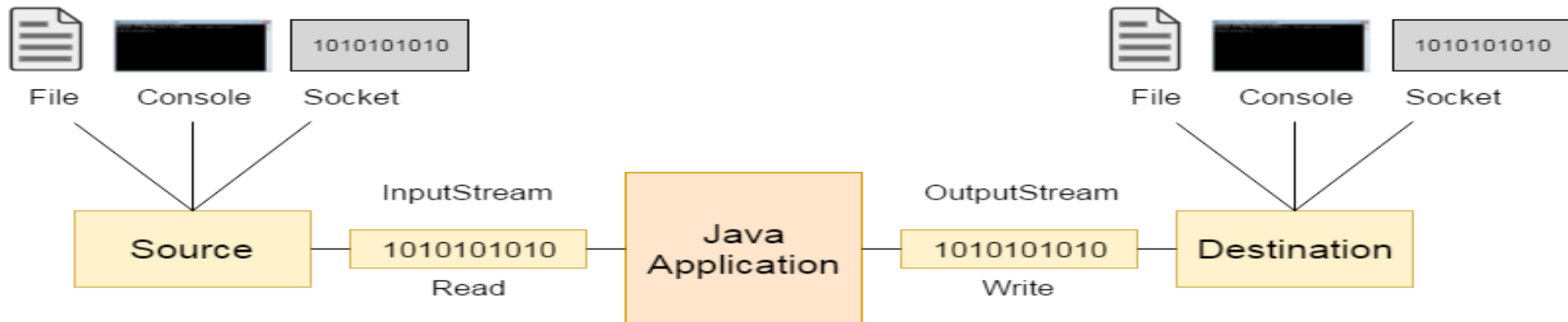
InputStream

Java application uses an input stream to read data from a source, it may be a file, an array, peripheral device or socket.

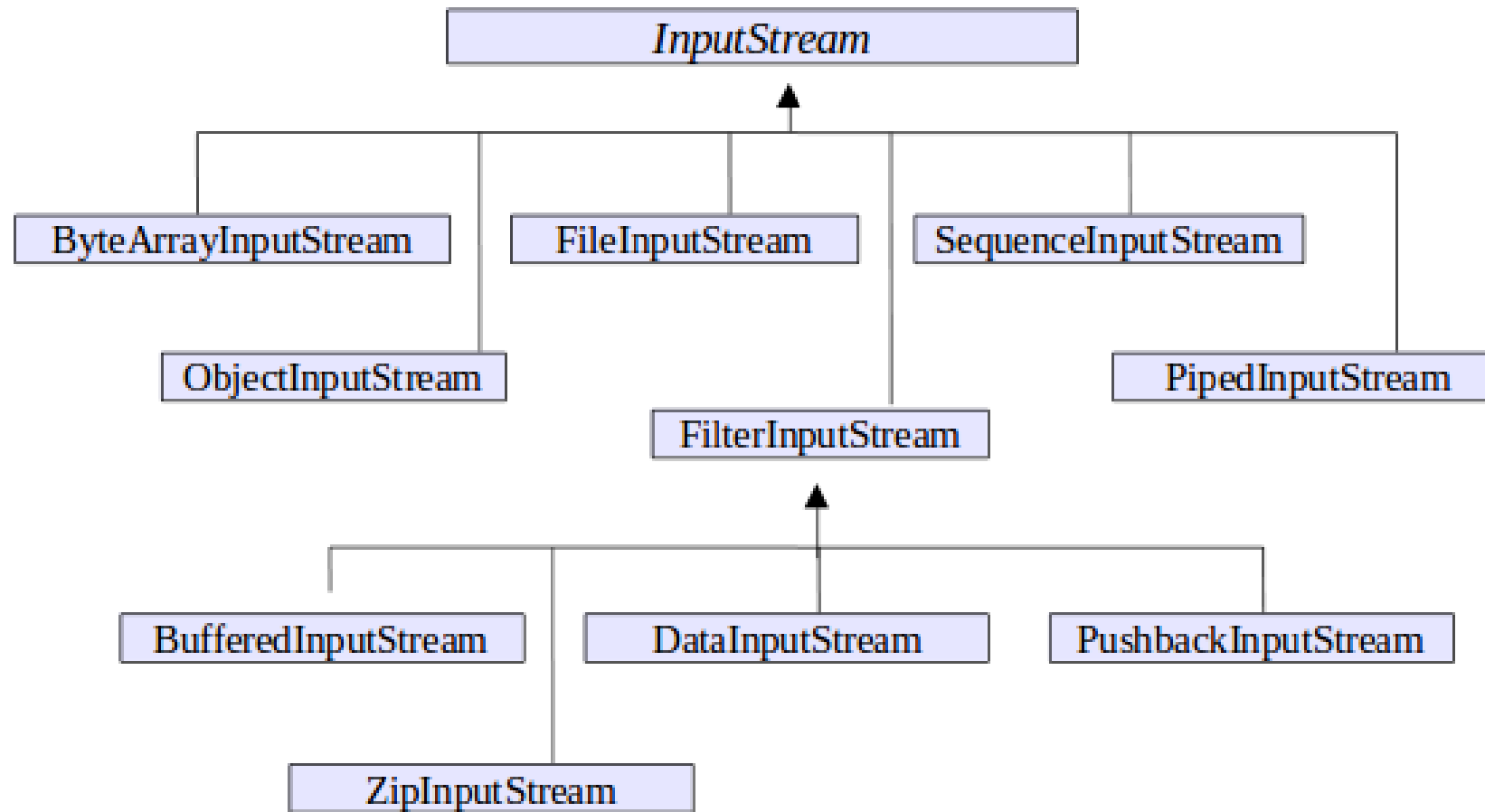
OutputStream

Java application uses an output stream to write data to a destination, it may be a file, an array, peripheral device or socket.

Let's understand working of Java OutputStream and InputStream by the figure given below.



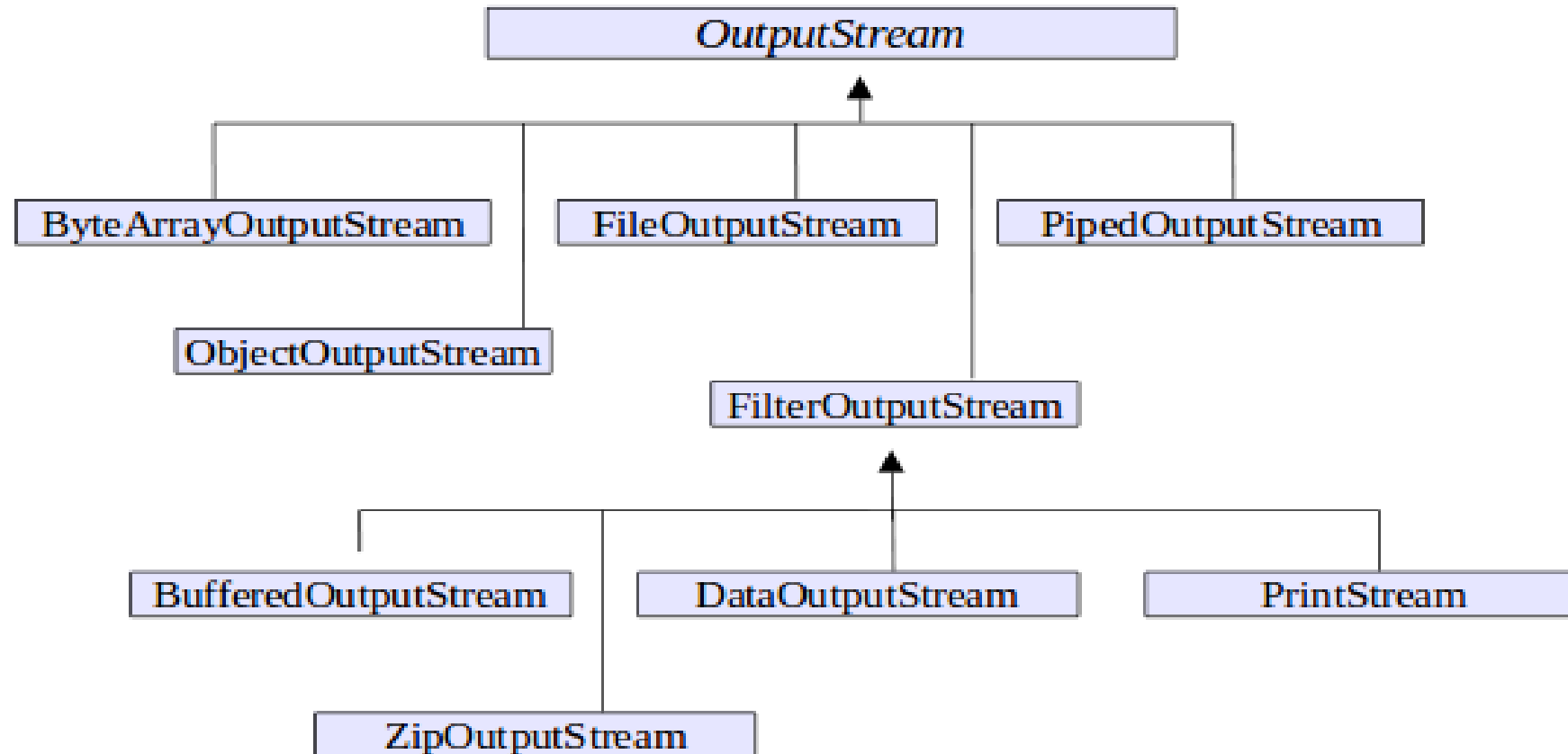
Byte Stream Classes-Input Stream



Common Method in InputStream

Method	Description
<code>public abstract <u>int</u> read() throws IOException</code>	reads the next byte of data from the input stream. It returns -1 at the end of file.
<code>public <u>int</u> available() throws IOException</code>	returns an estimate of the number of bytes that can be read from the current input stream.
<code>public void close() throws IOException</code>	is used to close the current input stream.

Byte Stream Classes-Output Stream



Common Method in OutputStream

Method	Description
1) <code>public void write(int) throws IOException</code>	is used to write a byte to the current output stream.
2) <code>public void write(byte[]) throws IOException</code>	is used to write an array of byte to the current output stream.
3) <code>public void flush() throws IOException</code>	flushes the current output stream.
4) <code>public void close() throws IOException</code>	is used to close the current output stream.

Methods in InputStream



- 1 int available() This method returns an estimate of the number of bytes that can be read (or skipped over) from this input stream without blocking by the next invocation of a method for this input stream.
- 2 void close() This method closes this input stream and releases any system resources associated with the stream.
- 3 void mark(int readlimit) This method marks the current position in this input stream.
- 4 boolean markSupported() This method tests if this input stream supports the mark and reset methods.
- 5 abstract int read() This method reads the next byte of data from the input stream.

Methods in InputStream Con"

6 int read(byte[] b) This method reads some number of bytes from the input stream and stores them into the buffer array b

7 int read(byte[] b, int off, int len) This method reads up to len bytes of data from the input stream into an array of bytes.

8 void reset() This method repositions this stream to the position at the time the mark method was last called on this input stream.

9 long skip(long n) This method skips over and discards n bytes of data from this input stream.

Methods in OutputStream

- 1 void close() This method closes this output stream and releases any system resources associated with this stream.
- 2 void flush() This method flushes this output stream and forces any buffered output bytes to be written out.
- 3 void write(byte[] b) This method writes *b.length* bytes from the specified byte array to this output stream.
- 4 void write(byte[] b, int off, int len) This method writes *len* bytes from the specified byte array starting at offset *off* to this output stream.
- 5 abstract void write(int b) This method writes the specified byte to this output stream.

BufferedInputStream Example

```
// Java program to demonstrate working of BufferedInputStream
import java.io.BufferedInputStream;
import java.io.FileInputStream;
import java.io.IOException;

// Java program to demonstrate BufferedInputStream methods
class BufferedInputStreamDemo
{
    public static void main(String args[]) throws IOException
    {
        // attach the file to FileInputStream
        FileInputStream fin = new FileInputStream("file1.txt");

        BufferedInputStream bin = new BufferedInputStream(fin);

        // illustrating available method
        System.out.println("Number of remaining bytes:" + bin.available());

        // illustrating markSupported() and mark() method
        boolean b=bin.markSupported();
        if (b)
            bin.mark(bin.available());
        // illustrating skip method

        /*Original File content:
        * This is my first line
        * This is my second line*/
        bin.skip(4);
```

```
// illustrating skip method
/*Original File content:
* This is my first line
* This is my second line*/
bin.skip(4);
System.out.println("FileContents :");

// read characters from FileInputStream and
// write them
int ch;
while ((ch=bin.read()) != -1)
    System.out.print((char)ch);

// illustrating reset() method
bin.reset();
while ((ch=bin.read()) != -1)
    System.out.print((char)ch);

// close the file
fin.close(); } }
```

Output: Number of remaining bytes:47

FileContents : is my first line This is my second line This is my first line This is my second line

BufferedOutputStream Example

```
//Java program demonstrating BufferedOutputStream

import java.io.*;

class BufferedOutputStreamDemo
{
    public static void main(String args[])throws Exception
    {
        FileOutputStream fout = new FileOutputStream("f1.txt");

        //creating bufferedOutputStream obj
        BufferedOutputStream bout = new BufferedOutputStream(fout);

        //illustrating write() method
        for(int i = 65; i < 75; i++)

        {
            bout.write(i);
        }

        byte b[] = { 75, 76, 77, 78, 79, 80 };
        bout.write(b);

        //illustrating flush() method
        bout.flush();

        //illustrating close() method
        bout.close();
        fout.close();
    } }
```

Output

ABCDEFGHIJKLMNOP

ByteArrayInputStream

The ByteArrayInputStream is composed of two words: ByteArray and InputStream.

As the name suggests, it can be used to read byte array as input stream.

Java ByteArrayInputStream class contains an internal buffer which is used to **read byte array** as stream.

In this stream, the data is read from a byte array.

The buffer of ByteArrayInputStream automatically grows according to data.

Java ByteArrayInputStream class constructors



Constructor	Description
<code>ByteArrayInputStream(byte[] ary)</code>	Creates a new byte array input stream which uses ary as its buffer array.
<code>ByteArrayInputStream(byte[] ary, int offset, int len)</code>	Creates a new byte array input stream which uses ary as its buffer array that can read up to specified len bytes of data from an array.

```
import java.io.*;

public class ReadExample
{
    public static void main(String[] args) throws IOException
    {
        byte[] buf = { 35, 36, 37, 38 };           // Create the new byte array input stream
        ByteArrayInputStream byt = new ByteArrayInputStream(buf);
        int k = 0;
        while ((k = byt.read()) != -1)
        {
            //Conversion of a byte into character
            char ch = (char) k;
            System.out.println("ASCII value of Character is:" + k + "; Special character is: " + ch);
        }
    }
}
```

Topic 1
Streams

Topic 2
Byte Streams

Topic 3
Buffered Stream

Third Lesson Summary

Use Byte streams for reading and writing

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream

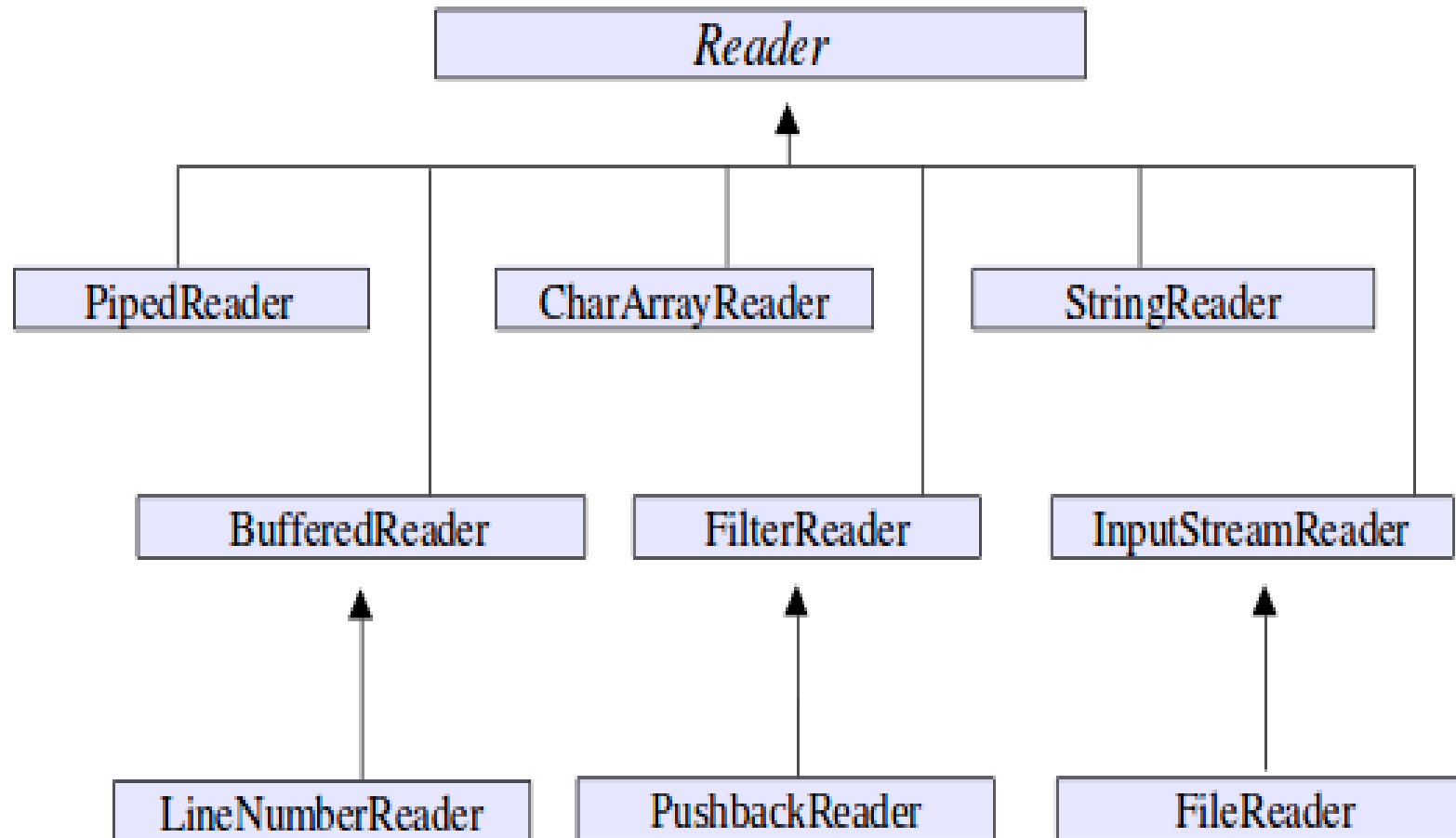


Lesson 5. Abstract and Final

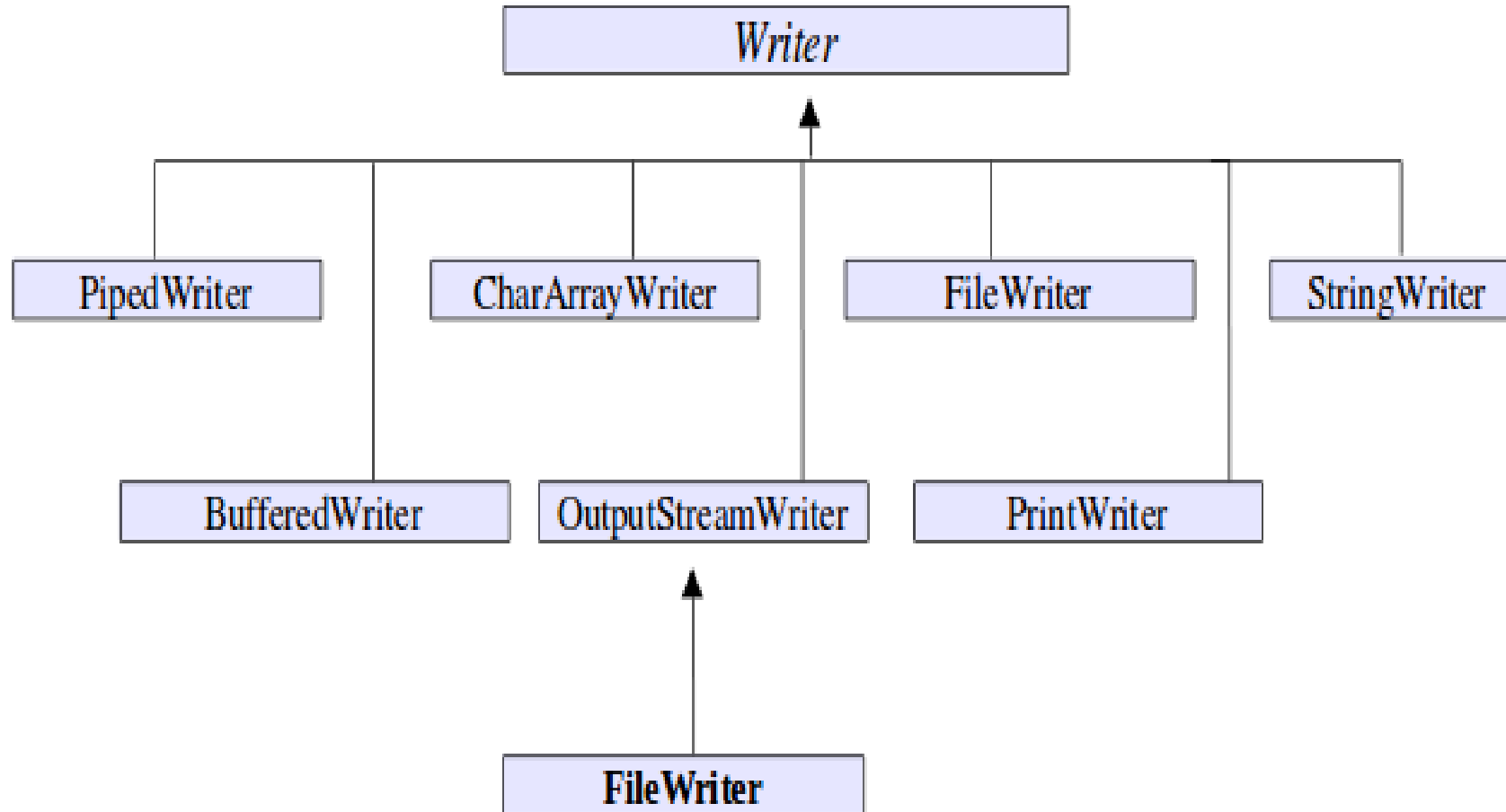
Character Streams

Stream Class	Meaning
BufferedReader	Buffered input character stream
BufferedWriter	Buffered output character stream
CharArrayReader	Input stream that reads from a character array
CharArrayWriter	Output stream that writes to a character array
FileReader	Input stream that reads from a file
FileWriter	Output stream that writes to a file
FilterReader	Filtered reader
FilterWriter	Filtered writer
InputStreamReader	Input stream that translates bytes to characters
LineNumberReader	Input stream that counts lines
OutputStreamWriter	Output stream that translates characters to bytes
PipedReader	Input pipe
PipedWriter	Output pipe
PrintWriter	Output stream that contains print() and println()
PushbackReader	Input stream that allows characters to be returned to the input stream
Reader	Abstract class that describes character stream input
StringReader	Input stream that reads from a string
StringWriter	Output stream that writes to a string
Writer	Abstract class that describes character stream output

Character Stream Classes: Reader



Character Stream Classes: Writer



BufferedInputStream

Java BufferedInputStream class is used to read information from stream.

It internally uses buffer mechanism to make the performance fast.

The important points about BufferedInputStream are:

- When the bytes from the stream are skipped or read, the internal buffer automatically refilled from the contained input stream, many bytes at a time.
- When a BufferedInputStream is created, an internal buffer array is created.

Methods



Method	Description
<code>int available()</code>	It returns an estimate number of bytes that can be read from the input stream without blocking by the next invocation method for the input stream.
<code>int read()</code>	It read the next byte of data from the input stream.
<code>int read(byte[] b, int off, int ln)</code>	It read the bytes from the specified byte-input stream into a specified byte array, starting with the given offset.
<code>void close()</code>	It closes the input stream and releases any of the system resources associated with the stream.
<code>void reset()</code>	It repositions the stream at a position the mark method was last called on this input stream.
<code>void mark(int readlimit)</code>	It sees the general contract of the mark method for the input stream.
<code>long skip(long x)</code>	It skips over and discards x bytes of data from the input stream.
<code>boolean markSupported()</code>	It tests for the input stream to support the mark and reset methods.

Java BufferedOutputStream Class



Java BufferedOutputStream class is used for buffering an output stream.

It internally uses buffer to store data.

It adds more efficiency than to write data directly into a stream. So, it makes the performance fast.

For adding the buffer in an OutputStream, use the BufferedOutputStream class.

Let's see the syntax for adding the buffer in an OutputStream:

```
OutputStream os= new BufferedOutputStream(new FileOutputStream("D:\\IO Package\\test  
out.txt"));
```

Reading Console Input



- Console input is accomplished by reading from **System.in**.
- To obtain a character-based stream that is attached to the console, wrap **System.in** in a **BufferedReader** object. **BufferedReader** supports a buffered input stream.
- A commonly used constructor is shown here:
BufferedReader(Reader inputReader)
- Here, **inputReader** is the stream that is linked to the instance of **BufferedReader** that is being created.
- **Reader** is an abstract class. One of its concrete subclasses is **InputStreamReader**, which converts bytes to characters.

Reading Console Input

- To obtain an `InputStreamReader` object that is linked to `System.in`, use the following constructor:

`InputStreamReader(InputStream inputStream)`

- Because `System.in` refers to an object of type `InputStream`, it can be used for `inputStream`.
- The following line of code creates a `BufferedReader` that is connected to the keyboard:

`BufferedReader br = new BufferedReader(new InputStreamReader(System.in));`

- After this statement executes, `br` is a character-based stream that is linked to the console through `System.in`.

Using BufferedReader to read characters from the console

```
import java. io .*;
class BRRead {
public static void main( String args []) throws IOException
{
char c;

Buffered Reader br = new Buffered Reader ( new InputStream Reader ( System . in));

System . out. println (" Enter characters , ' q' to quit.");
do {

c = ( char) br. read (); System . out. println ( c);
} while( c != ' q');
} }
```

Read a string from console using a BufferedReader

```
import java. io .*;
class BRReadLines {
public static void main( String args []) throws IOException
{
Buffered Reader br = new Buffered Reader ( new InputStream Reader ( System . in ));
String str;
System . out. println (" Enter lines of text.");
System . out. println (" Enter ' stop ' to quit.");
do {
str = br. readLine (); System . out. println ( str );
} while (! str. equals(" stop"));
} }
```

Read a number from console using a BufferedReader

```
import java. io .*;
public class UserInputInteger
{
public static void main( String args []) throws IOException
{
InputStream Reader read = new InputStream Reader ( System . in );
BufferedReader in = new BufferedReader ( read );
int number;

System . out. println (" Enter the number");
number = Integer. parseInt( in. readLine ());

System . out. println ( number );
}
}
```

Writing Console Output

```
class WriteDemo {  
    public static void main( String args [])  
    {  
        int b;  
        b = ' A';  
  
        System . out. write( b);  
  
        System . out. write('\ n');  
    }  
}
```

```
import java. io .*;  
public class PrintWriterDemo {  
    public static void main( String args []) {  
        PrintWriter pw = new PrintWriter ( System . out .true );  
        pw. println (" This is a string ");  
        int i = -7;  
        pw. println (i);  
        double d = 4.5 e -7;  
        pw. println ( d);  
    }  
}
```

Reading and Writing Files

- Two of the most often-used stream classes are **FileInputStream** and **FileOutputStream**, which create byte streams linked to files.
- To open a file, you simply create an object of one of these classes, specifying the name of the file as an argument to the constructor.
- Although both classes support additional constructors, the following are the forms that we will be using:

FileInputStream(String fileName) throws FileNotFoundException

FileOutputStream(String fileName) throws FileNotFoundException

Reading and Writing Files - Example

```
import java. io .*;
class CopyFile {

public static void main( String args []) throws IOException
{

int i; FileInputStream fin;

FileOutputStream fout;

try {
// open input file
try {
} fin = new FileInputStream ( args [0]);
catch ( FileNotFoundException e)
{ System . out. println (" Input File Not Found ");
return ; }
```

Reading and Writing Files – Example Contd.

```
try {  
    fout = new FileOutputStream ( args [1]);  
  
} catch ( FileNotFoundException e) { System . out. println (" Output File Error"); return ;  
}  
  
} catch ( Array Index Out OfBounds Exception e)  
{ System . out. println (" Usage: CopyFile From To");  
return ;}  
try {  
    do {  
        i = fin. read ();  
        if( i != -1) fout. write( i);  
        } while( i != -1);  
  
    } catch ( IOException e) { System . out. println (" File Error");  
    }  
fin. close ();  
fout. close ();  
}}
```

```
import java.io.FileInputStream;

public class DataStreamExample
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin=new FileInputStream("D:\\testout.txt");

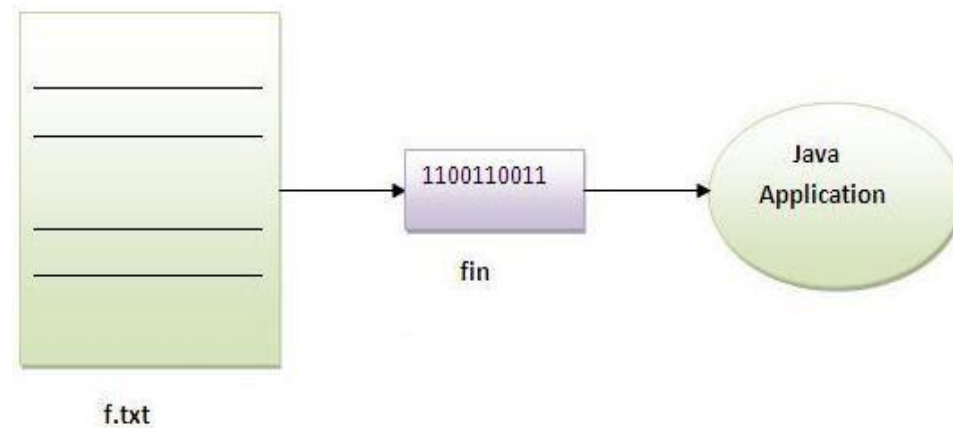
            int i=fin.read();

            System.out.print((char)i);

            fin.close();

        }
        catch(Exception e)
        {
            System.out.println(e);

        }
    }
}
```



Note: Before running the code, a text file named as "**testout.txt**" is required to be created. In this file, we are having following content:

Hello welcomes you all for java programming.

After executing the above program, you will get a single character from the file which is 75 (in byte form).

To see the text, you need to convert it into character.

Output:

H

```
import java.io.*;
class SimpleRead
{
    public static void main(String args[])
    {
        try
        {
            FileInputStream fin=new FileInputStream("abc.txt");
            int i=0;
            while((i=fin.read())!=-1)
            {
                System.out.println((char)i);
            }
            fin.close();
        }
        catch(Exception e)    { system.out.println(e);    } } }
```

Output:
**Hello welcomes you all for java
programming.**

Topic 1
Character Streams

Topic 2
Reading & Writing Console I/O

Topic 3
BufferedReader

Topic 4
Reading and Writing Files

Fourth Lesson Summary

Use BufferedReader, Character and File Streams for reading and writing

Course Progress (use before starting new lesson)



Lesson 1. Exceptions



Lesson 2. Built in and Custom Exceptions



Lesson 3. Byte Stream



Lesson 4. Character Stream



Lesson 5. Abstract and Final

Abstract methods

- you can declare a *method* without defining it:

```
public abstract void draw(int size);
```

 - Notice that the body of the method is missing
- A method that has been declared but not defined is an abstract method

Abstract classes I

- Any class containing an abstract method is an abstract class
- You must declare the class with the keyword `abstract`:

```
abstract class MyClass {...}
```

- An abstract class is *incomplete*
 - It has “missing” method bodies
- You cannot instantiate (create a new instance of) an abstract class

Abstract classes II

- You can extend (subclass) an abstract class
 - If the subclass defines all the inherited abstract methods, it is “complete” and can be instantiated
 - If the subclass does *not* define all the inherited abstract methods, it too must be abstract
- You can declare a class to be abstract even if it does not contain any abstract methods
 - This prevents the class from being instantiated

Why have abstract classes?

- Suppose you wanted to create a class Shape, with subclasses Oval, Rectangle, Triangle, Hexagon, etc.
- You don't want to allow creation of a "Shape"
 - Only *particular* shapes make sense, not *generic* ones
 - If Shape is abstract, you can't create a new Shape
 - You *can* create a new Oval, a new Rectangle, etc.
- Abstract classes are good for defining a general category containing specific, "concrete" classes

An example abstract class

- public abstract class Animal
- {
 abstract int eat();
 abstract void breathe();
}
- This class cannot be instantiated
- Any non-abstract subclass of Animal must provide the eat() and breathe() methods

EXAMPLE

- `class Shape { ... }`
- `class Star extends Shape {
 void draw() { ... }
 ...
}`
- `class Crescent extends Shape {
 void draw() { ... }
 ...
}`
- `Shape someShape = new Star();`
 - This is legal, because a Star *is* a Shape
- `someShape.draw();`
 - This is a syntax error, because *some* Shape might not have a `draw()` method
 - Remember: ***A class knows its superclass, but not its subclasses***

Solution

- abstract class Shape {
 void draw();
}
- class Star extends Shape {
 void draw() { ... }
 ...
}
- class Crescent extends Shape {
 void draw() { ... }
 ...
}
- Shape someShape = new Star();
 - This is legal, because a Star *is* a Shape
 - However, Shape someShape = new Shape(); is *no longer* legal
- someShape.draw();
 - This is legal, because every actual instance *must* have a draw() method

Final Keyword



- The **final keyword** in java is used to restrict the user.
- The java final keyword can be used in many context.
- Final can be:
 - variable
 - method
 - class

- The final keyword can be applied with the variables, a final variable that have no value it is called blank final variable or uninitialized final variable.
- It can be initialized in the constructor only.
- The blank final variable can be static also which will be initialized in the static block only.
- Let's first learn the basics of final keyword.

Java final variable

- If you make any variable as final, you cannot change the value of final variable(It will be constant).
- **Example of final variable**
 - There is a final variable speed limit, we are going to change the value of this variable, but It can't be changed because final variable once assigned a value can never be changed.

EXAMPLE

```
class Bike9
{
    final int speedlimit=90; //final variable
    void run()
    {
        speedlimit=400;
    }
    public static void main(String args[])
    {
        Bike9 obj=new Bike9();
        obj.run();
    }
} //end of class
```

Java final method

- If you make any method as final, you cannot override it.

```
class Bike
```

```
{  
    final void run()  
    {  
        System.out.println("running");  
    }  
}
```

```
class Honda extends Bike
```

```
{  
    void run()  
    {System.out.println("running safely with 100kmph");}  
    public static void main(String args[])  
    {  
        Honda honda= new Honda();  
        honda.run();  
    }  
}
```

Java final class

- If you make any class as final, you cannot extend it.

```
final class Bike
```

```
{  
}
```

```
class Honda1 extends Bike
```

```
{
```

```
    void run()
```

```
{
```

```
        System.out.println("running safely with 100kmph");
```

```
}
```

```
    public static void main(String args[])
```

```
{
```

```
        Honda1 honda= new Honda();
```

```
        honda.run();
```

```
}
```

```
}
```

Is final method inherited?

- Yes, final method is inherited but you cannot override it.

```
class Bike
```

```
{
```

```
    final void run()
```

```
{
```

```
        System.out.println("running...");
```

```
}
```

```
}
```

```
class Honda2 extends Bike
```

```
{
```

```
    public static void main(String args[])
```

```
{
```

```
        new Honda2().run();
```

```
}
```

```
}
```

What is blank or uninitialized final variable?



- A final variable that is not initialized at the time of declaration is known as blank final variable.
- If you want to create a variable that is initialized at the time of creating object and once initialized may not be changed, it is useful.
- For example PAN CARD number of an employee.
- It can be initialized only in constructor.

Can we initialize blank final variable?

Yes, but only in constructor.

```
class Bike10
{
    final int speedlimit; // blank final variable
    Bike10()
    {
        speedlimit=70;
        System.out.println(speedlimit);
    }
    public static void main(String args[])
    {
        new Bike10();
    }
}
```

static blank final variable

- A static final variable that is not initialized at the time of declaration is known as static blank final variable.
- It can be initialized only in static block.

```
class A
{
    static final int data; //static blank final variable
    static
    {
        data=50;
    }
    public static void main(String args[])
    {
        System.out.println(A.data);
    }
}
```

What is final parameter?

- If you declare any parameter as final, you cannot change the value of it.

```
class Bike11
{
    int cube(final int n)
    {
        n=n+2;//can't be changed as n is final
        n*n*n;
    }
    public static void main(String args[])
    {
        Bike11 b=new Bike11();
        b.cube(5);
    }
}
```

Can we declare a constructor final?

- No, because constructor is never inherited.

Topic 1
Abstract class

Topic 2
final keyword

Fifth Lesson Summary

Working of abstract and final keyword

Thank You!

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

1. Project Definition

Introduction

A **project** is a planned set of activities carried out to achieve a specific objective within a defined **scope, time, cost, and quality**.

In medical device development, a project may involve designing, developing, testing, validating, obtaining regulatory clearance/approval, and launching a new medical device.

Example

Development of an **IoT-based patient monitoring system** can be considered a project with:

- Objective – Develop a real-time patient monitoring system
- Duration – 12 months
- Budget – ₹20 lakh
- Team – Biomedical engineers, software engineers, clinicians and marketing personnel
- Deliverable – Tested and validated prototype/product

Major elements of project definition

1. **Project title**
2. **Problem statement**
3. **Objectives**
4. **Scope**
5. **Deliverables**
6. **Target customers**
7. **Timeline**
8. **Budget**
9. **Resources**
10. **Quality requirements**
11. **Regulatory requirements**
12. **Risk management**

Importance

A clear project definition helps the team understand **what has to be developed, why it is required, who will use it, how much it will cost, and when it should be completed**.

MEDICAL DEVICE MARKETING
UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

2. Financial Targets

Financial targets are the **specific financial goals established for a product or project**.

They help determine whether the product is commercially viable.

Important financial targets

1. Development budget

Maximum amount allocated for product development.

2. Manufacturing cost

Cost required to manufacture one unit.

3. Selling price

Price at which the product is sold.

4. Revenue target

Expected income from product sales.

5. Profit target

Expected profit from the product.

6. Break-even point

The sales level at which total revenue equals total cost.

7. Return on Investment (ROI)

Measures the return generated from an investment.

$$\text{ROI} = (\text{Net Return} / \text{Investment}) \times 100$$

Example

Suppose:

- Manufacturing cost = ₹8,000/device
- Selling price = ₹15,000/device
- Expected sales = 1,000 devices

The company can estimate its revenue, contribution margin, operating costs, and expected profit before deciding whether to proceed with the project.

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

3. Physician Input

Introduction

Physician input refers to obtaining information, suggestions, feedback, and clinical requirements from doctors during medical product development.

Physicians are important stakeholders because they understand the **clinical problem, patient requirements, workflow, treatment procedures, and limitations of existing technologies.**

Importance of Physician Input

1. Identification of clinical problems

Physicians can identify problems that require technological solutions.

Example:

A physician may identify that existing patient monitoring systems provide information too late for early intervention.

2. Defining product requirements

Doctors can specify:

- Required parameters
- Accuracy
- Response time
- Clinical functionality
- Patient population
- Desired alerts

3. Product design

Physician feedback helps engineers develop a device suitable for actual clinical environments.

4. Usability

Doctors can evaluate:

- User interface
- Display
- Controls
- Workflow
- Ease of operation

5. Clinical validation

Physicians may participate in clinical evaluation and provide feedback regarding clinical performance.

6. Risk identification

Doctors can identify possible clinical hazards and inappropriate uses.

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

7. Product acceptance

A product developed with appropriate clinical input is more likely to be accepted by healthcare professionals.

Methods of Obtaining Physician Input

- Interviews
- Questionnaires
- Focus groups
- Clinical observations
- Product demonstrations
- Advisory panels
- Prototype evaluation
- Clinical studies

Example

For a **smart prosthetic limb**, engineers may develop the sensors and control system, while physicians and rehabilitation professionals can provide input regarding:

- Patient comfort
- Gait requirements
- Safety
- Socket fit
- Rehabilitation needs
- Clinical usability

4. FDA Approval

Introduction

The **U.S. Food and Drug Administration (FDA)** regulates medical devices marketed in the United States. FDA's Center for Devices and Radiological Health (CDRH) regulates firms that manufacture, repack, relabel, or import medical devices for the U.S. market.

An important point for students is that **not every medical device receives an "FDA approval."** Depending on the device and regulatory pathway, the product may be **510(k) cleared, De Novo classified, PMA approved, or exempt from a premarket submission.**

Objectives of FDA Regulation

FDA regulation aims to ensure that medical devices:

- Are reasonably safe
- Are effective for their intended use

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Meet applicable regulatory requirements
- Are appropriately manufactured
- Have appropriate labeling
- Continue to be monitored after marketing

Step 1 – Identify the Intended Use

The first step is to clearly define:

- What the device does
- Who will use it
- Which patients will use it
- What disease/condition it addresses
- Whether it diagnoses, treats, prevents, monitors, or supports a condition

The intended use and indications for use are important because they influence the device's classification and regulatory pathway.

Step 2 – Determine Device Classification

FDA generally classifies medical devices into:

Class I – Low risk

Generally subject to **General Controls**.

Examples may include certain low-risk devices.

Class II – Moderate risk

Subject to **General Controls + Special Controls**.

Class III – High risk

Subject to **General Controls + Premarket Approval (PMA)** in applicable cases.

FDA states that classification is risk-based, with Class I generally representing the lowest risk and Class III the highest.

Class	Risk	Main Controls
Class I	Low	General Controls
Class II	Moderate	General + Special Controls
Class III	High	General Controls + PMA in applicable cases

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Step 3 – Identify the Appropriate Regulatory Pathway

The major pathways students should know are:

A. 510(k) – Premarket Notification

The manufacturer generally demonstrates that the new device is **substantially equivalent** to a legally marketed predicate device.

The comparison generally considers:

- Intended use
- Technological characteristics
- Performance
- Safety

FDA explains that a 510(k) is a pathway based on substantial equivalence to an appropriate legally marketed device.

Important: The correct term is generally "**FDA clearance**", not "FDA approval," for a 510(k).

B. De Novo Classification

The **De Novo pathway** can be used for certain novel devices for which there is no suitable predicate but whose risks can be appropriately controlled.

A successful De Novo request establishes a new classification for that device type. FDA identifies De Novo as a pathway for certain novel device types that have not previously been marketed in the United States.

C. Premarket Approval – PMA

PMA is the most stringent major premarket pathway.

It is generally associated with **Class III devices** requiring PMA.

The manufacturer must provide scientific evidence supporting the safety and effectiveness of the device. FDA describes PMA as the scientific and regulatory review used to evaluate the safety and effectiveness of applicable Class III devices.

Examples of devices that may require PMA include certain implantable and life-supporting devices.

D. HDE

Humanitarian Device Exemption (HDE) is associated with certain devices intended to benefit patients with rare diseases or conditions meeting the applicable statutory criteria.

Step 4 – Preclinical/Nonclinical Testing

Before clinical use or marketing, applicable devices may require testing such as:

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Mechanical testing
- Electrical safety testing
- Biocompatibility
- Software verification
- Electromagnetic compatibility
- Sterilization validation
- Packaging testing
- Performance testing
- Animal testing where appropriate

The exact testing depends on the device.

Step 5 – Clinical Evaluation/Investigation

For devices requiring clinical evidence, clinical investigations may be conducted to evaluate:

- Safety
- Effectiveness
- Performance
- Patient outcomes
- Adverse events

Where applicable, an **Investigational Device Exemption (IDE)** permits certain investigational devices to be used in clinical studies in accordance with FDA requirements.

Step 6 – Quality Management System

Manufacturers must establish an appropriate quality system.

A major current development is that FDA's **Quality Management System Regulation (QMSR)** became effective on **February 2, 2026**. QMSR amended 21 CFR Part 820 and incorporates by reference **ISO 13485:2016** for medical device quality management systems.

Therefore, students should now understand:

21 CFR Part 820 → QMSR → incorporates ISO 13485:2016

The QMS supports areas such as:

- Design and development
- Manufacturing
- Quality control
- Documentation

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Corrective and preventive actions
- Complaint handling
- Risk management
- Records

Step 7 – Prepare Regulatory Submission

The manufacturer prepares the appropriate regulatory submission.

Depending on the pathway, this may include:

- Device description
- Intended use
- Indications for use
- Design information
- Risk information
- Performance data
- Test results
- Clinical data where required
- Manufacturing information
- Quality-system information
- Labeling

Step 8 – Labeling

Medical device labeling is an important regulatory requirement.

FDA's general device labeling requirements are contained in **21 CFR Part 801**.

Labeling may include:

- Device name
- Manufacturer information
- Intended use
- Indications
- Instructions for use
- Warnings
- Precautions
- Contraindications

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Installation instructions
- Maintenance information

Labeling must not provide misleading information or unsupported claims.

Step 9 – FDA Review

FDA reviews the submitted information according to the applicable pathway.

The review may involve:

- Technical evaluation
- Clinical evaluation
- Statistical evaluation
- Manufacturing/quality considerations
- Labeling review
- Risk assessment

FDA may request additional information where necessary.

Step 10 – Regulatory Decision

Depending on the pathway, the outcome may be:

- 510(k) clearance
- De Novo classification
- PMA approval
- HDE approval
- Exemption where applicable

Step 11 – Post-Market Requirements

FDA oversight does not end when a device reaches the market.

Post-market activities may include:

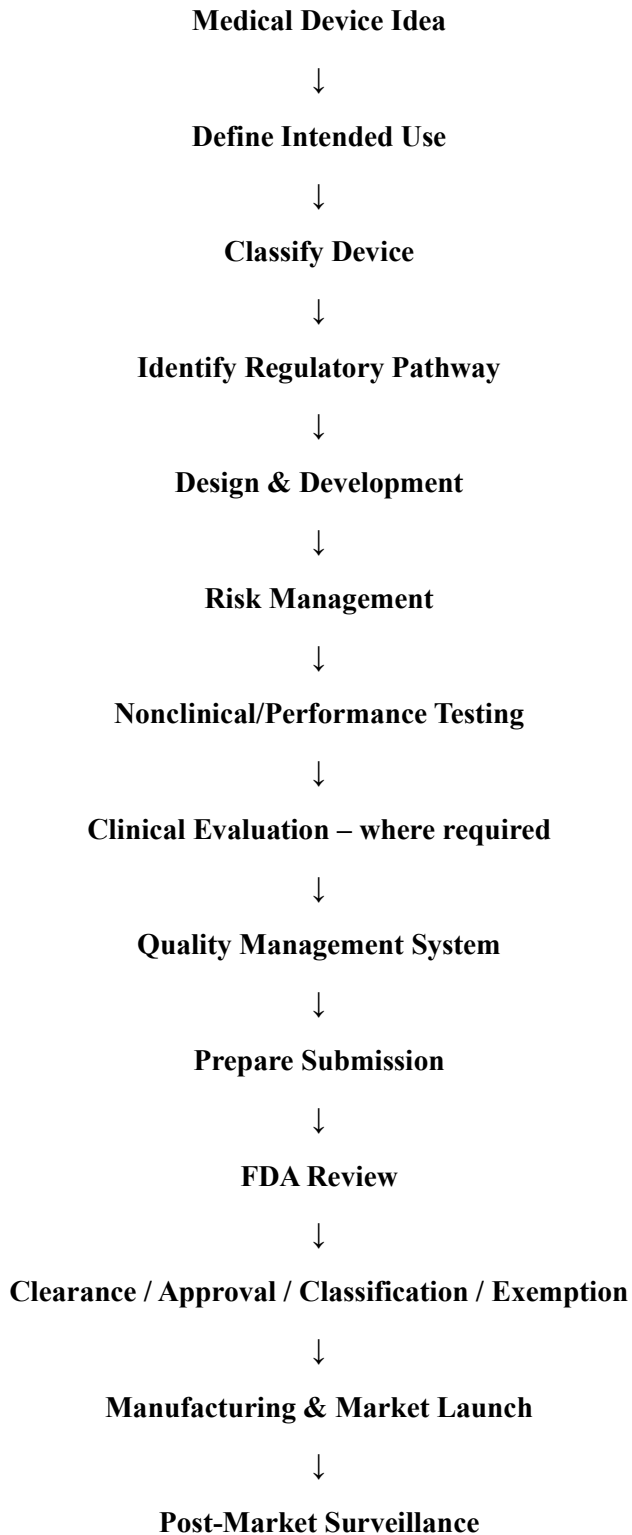
- Complaint handling
- Medical device reporting
- Corrective actions
- Recalls where necessary
- Post-market surveillance
- Manufacturing quality monitoring

MEDICAL DEVICE MARKETING UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Monitoring of device safety and performance

FDA states that it is responsible for assuring that medical devices available in the United States remain safe and effective throughout their total product lifecycle.

Simplified FDA Pathway



MEDICAL DEVICE MARKETING
UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Important Difference: Clearance vs Approval

Term	Common pathway	Meaning
FDA Clearance	510(k)	Device has been determined substantially equivalent to an appropriate predicate
FDA Approval	PMA	FDA has approved the PMA based on applicable evidence of safety and effectiveness
De Novo Classification	De Novo	New device type classified through the De Novo pathway
Exempt	Certain devices	No applicable premarket submission, subject to the conditions and limitations of the exemption

5. Reimbursement

Introduction

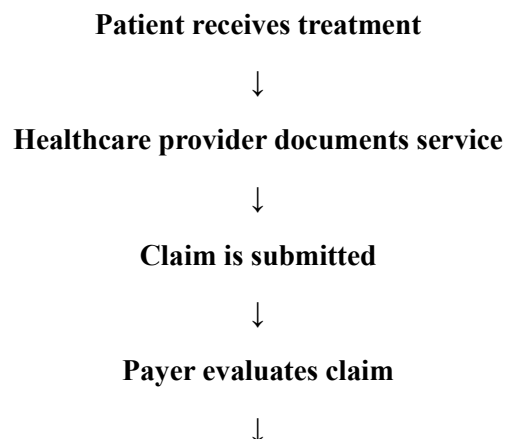
Reimbursement refers to the payment made to healthcare providers or organizations for eligible healthcare services, procedures, or products.

For a medical device company, reimbursement is important because a product may be clinically effective but commercially unsuccessful if customers cannot obtain adequate financial coverage for its use.

Major Participants

- Patient
- Doctor
- Hospital
- Insurance company/payer
- Government
- Medical device manufacturer

Basic Process



MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Claim approved/rejected



Payment made

Importance of Reimbursement

Reimbursement influences:

- Patient affordability
- Hospital purchasing
- Product adoption
- Market size
- Revenue
- Product pricing

Example

A new cardiac monitoring device may provide excellent clinical benefits. However, if hospitals or patients cannot recover the cost through an applicable payment mechanism, adoption may remain low.

6. Managing Issues

During product development and commercialization, several issues can arise.

Common issues

- Technical failures
- Component shortages
- Design changes
- Budget overruns
- Regulatory delays
- Testing failures
- Customer complaints
- Manufacturing problems
- Software errors
- Supplier delays

Issue Management Process

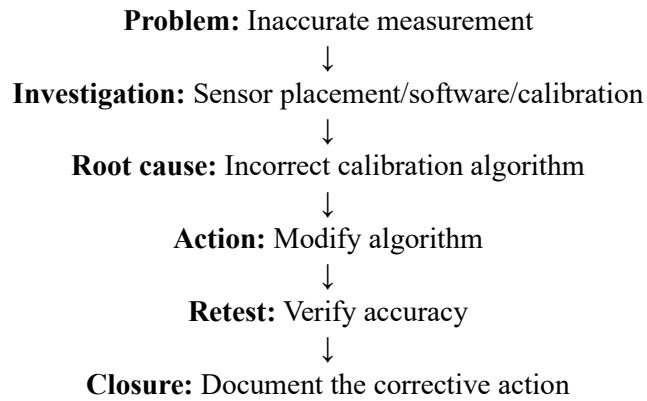
Identify issue → Analyze cause → Determine impact → Prioritize → Develop corrective action → Assign responsibility → Implement action → Monitor → Close issue

Example

If a prototype shows inaccurate SpO₂ readings:

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH



7. Meeting Deadlines

Meeting deadlines means completing project activities within the planned time.

Methods

1. Develop a project schedule

Define:

- Activities
- Start date
- End date
- Dependencies
- Responsible person

2. Prioritize critical activities

Identify activities that directly affect the final delivery date.

3. Monitor progress

Use:

- Gantt charts
- Milestone tracking
- Weekly meetings
- Progress reports

4. Allocate resources properly

Ensure availability of:

- Personnel
- Equipment
- Materials

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Budget

5. Manage risks

Identify potential delays before they occur.

6. Take corrective action

If an activity is delayed:

- Reallocate resources
- Change sequence
- Add manpower
- Revise schedule
- Address technical problems

Example

If clinical testing is delayed by two weeks, the team should assess whether documentation, software development, or manufacturing activities can proceed in parallel without compromising quality.

8. Product Launch

Introduction

Product launch is the process of introducing a new product into the target market and making it available to customers.

For a medical device, product launch is more than simply selling the product. It involves:

- Regulatory readiness
- Manufacturing readiness
- Market preparation
- Pricing
- Distribution
- Sales training
- Customer support
- Product communication
- Post-market monitoring

Stages of Product Launch

1. Market Preparation

Before launching the product, the company should understand:

- Target customers

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Market size
- Competitors
- Customer requirements
- Pricing
- Distribution channels
- Market barriers

2. Regulatory Readiness

The company must ensure that applicable regulatory requirements have been satisfied before marketing the device in the relevant jurisdiction.

This includes appropriate:

- Regulatory status
- Labeling
- Documentation
- Quality systems
- Manufacturing controls

3. Manufacturing Readiness

The company should ensure that sufficient quantities can be produced with consistent quality.

Activities include:

- Supplier qualification
- Production planning
- Quality inspection
- Packaging
- Sterilization where required
- Inventory planning

4. Pricing Strategy

The company determines the price based on:

- Manufacturing cost
- Competitor prices
- Customer willingness to pay
- Clinical value

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Distribution cost
- Market positioning
- Reimbursement environment

5. Sales Preparation

The sales team should understand:

- Product features
- Clinical benefits
- Product limitations
- Target customers
- Competitors
- Pricing
- Demonstration procedure
- Customer objections

6. Marketing and Promotion

Promotion may include:

- Product demonstrations
- Conferences
- Medical exhibitions
- Digital marketing
- Scientific publications
- Healthcare professional education
- Sales presentations

Medical device claims should remain consistent with applicable regulatory requirements and authorized/intended uses.

7. Distribution

The company determines how the product will reach customers.

Possible channels include:

Manufacturer → Distributor → Hospital → Patient

or

Manufacturer → Hospital

MEDICAL DEVICE MARKETING
UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

or

Manufacturer → Clinic → Patient

8. Product Training

Healthcare professionals may need training on:

- Installation
- Operation
- Maintenance
- Safety
- Troubleshooting
- Clinical use

9. Launch

The product is officially introduced to the target market.

Launch activities may include:

- Product announcement
- Demonstrations
- Distributor activation
- Sales campaigns
- Hospital visits
- Customer training

10. Post-Launch Monitoring

After launch, the company monitors:

- Sales
- Customer feedback
- Complaints
- Device failures
- Returns
- Adverse events
- Market acceptance
- Profitability

MEDICAL DEVICE MARKETING UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Product Launch Framework



Factors Determining Successful Product Launch

Technical readiness

The device should perform according to its specifications.

Clinical readiness

Healthcare professionals should understand its clinical value.

Regulatory readiness

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Required regulatory requirements must be satisfied.

Financial readiness

The company should have adequate financial resources.

Manufacturing readiness

Production should meet expected demand.

Sales readiness

Sales personnel should be properly trained.

Customer support

Installation, training, maintenance, and troubleshooting should be available.

Market readiness

There should be sufficient customer demand.

9. Forecasting

Forecasting is the process of **estimating future demand, sales, revenue, or market requirements**.

It helps companies determine how many products they should manufacture and how much inventory they should maintain.

Types of Forecasting

1. Qualitative forecasting

Based on expert opinion and market knowledge.

Methods include:

- Expert opinion
- Sales-force opinion
- Customer surveys
- Delphi method

2. Quantitative forecasting

Uses historical data and mathematical/statistical techniques.

Examples:

- Moving averages
- Trend analysis
- Regression analysis
- Time-series analysis

Factors Affecting Medical Device Forecasting

- Number of hospitals

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Patient population
- Disease prevalence
- Competitor products
- Product price
- Healthcare policies
- Reimbursement
- Technology adoption
- Economic conditions

Example

If a company sold:

- 500 units in Year 1
- 700 units in Year 2
- 900 units in Year 3

The company may use historical trends and market growth assumptions to estimate future demand.

Importance

Forecasting helps with:

- Production planning
- Inventory control
- Manpower planning
- Financial planning
- Sales targets
- Supply-chain management

10. Pricing

Pricing is the process of determining the **amount a customer must pay for a product or service.**

Objectives of Pricing

- Recover costs
- Generate profit
- Gain market share
- Compete effectively
- Reflect product value
- Encourage adoption

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Factors Affecting Pricing

1. Manufacturing cost

Higher production costs generally increase the minimum viable selling price.

2. Research and development cost

Medical device companies must recover development investment over time.

3. Competitor pricing

The company should understand prices of competing products.

4. Customer willingness to pay

Customers may pay more for significant clinical and economic benefits.

5. Clinical value

A product that significantly improves outcomes may justify a higher price.

6. Reimbursement

The available payment mechanism can strongly influence practical affordability and adoption.

7. Distribution cost

Distributor margins, transportation, installation, and service affect the final price.

Common Pricing Approaches

Cost-plus pricing:

Cost + desired profit margin

Competition-based pricing:

Price based on competitors

Value-based pricing:

Price based on perceived/ demonstrated customer and clinical value

Penetration pricing:

Lower initial price to encourage market adoption

Premium pricing:

Higher price for differentiated products with strong value or positioning

11. Product and Sales Support

Introduction

Product and sales support refers to the technical, clinical, marketing, training, and service assistance provided to customers and sales teams before and after a product is sold.

Product Support

Product support includes:

- Installation

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- User training
- Operation manuals
- Technical assistance
- Preventive maintenance
- Corrective maintenance
- Spare parts
- Software updates
- Troubleshooting
- Warranty service

Sales Support

Sales support helps sales personnel successfully communicate and demonstrate the product.

It includes:

- Product brochures
- Technical specifications
- Demonstration units
- Product presentations
- Training for sales personnel
- Competitor comparison
- Frequently asked questions
- Clinical evidence
- Customer case studies
- Pricing information

Importance

1. Improves customer confidence

Customers feel more confident when technical support is available.

2. Increases sales effectiveness

Sales personnel can answer technical and clinical questions.

3. Reduces product misuse

Proper training helps users operate devices correctly.

4. Improves customer satisfaction

Fast service and troubleshooting improve the customer experience.

5. Encourages repeat purchases

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

Satisfied customers are more likely to purchase additional products.

Example

For an ultrasound system, sales support may include product demonstration and clinical application training, while product support includes installation, preventive maintenance, troubleshooting, and service.

12. Managing an Existing Product Line

Introduction

A **product line** is a group of related products marketed by the same company.

For example, a medical technology company may have a product line containing:

- Basic patient monitor
- Advanced patient monitor
- ICU monitor
- Wireless monitor

Managing an existing product line means continuously monitoring and improving the products to maintain their **market position, profitability, quality, and customer satisfaction**.

Major Activities

1. Monitor sales performance

Track:

- Sales volume
- Revenue
- Profit
- Market share
- Growth rate

2. Monitor customer feedback

Collect information about:

- Product satisfaction
- Complaints
- Usability
- Reliability
- Desired improvements

3. Product improvement

Improve:

MEDICAL DEVICE MARKETING

UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Hardware
- Software
- User interface
- Accuracy
- Connectivity
- Reliability

4. Manage product variants

Companies may offer different versions for different customer segments.

5. Manage pricing

Prices may need adjustment based on:

- Competition
- Cost
- Demand
- Market position

6. Inventory management

Maintain sufficient stock without creating excessive inventory.

7. Competitor monitoring

Track:

- New competitor products
- Technology changes
- Pricing
- Features
- Market strategy

8. Product life-cycle management

A product generally passes through:

Introduction → Growth → Maturity → Decline

At each stage, the company may modify its marketing and product strategy.

9. Decide whether to modify or discontinue

If a product has declining demand, high maintenance costs, or is replaced by better technology, the company may:

- Improve it
- Reposition it
- Reduce its price

MEDICAL DEVICE MARKETING
UNIT – II: PRODUCT DEVELOPMENT AND LAUNCH

- Replace it
- Discontinue it